



O M N I R A

INTELLIGENCE GRAPH

Architecture, Motion and Live Operational Experience

A Product, Visual, Runtime, Accessibility, Performance
and Delivery Architecture for Omnira

CANONICAL
FOUNDER APPROVED

Version 1.0 · Juli 2026 · Internt arkitekturdokument

OMNIRA

OMNIRA INTELLIGENCE GRAPH

Architecture, Motion and Live Operational Experience

A Product, Visual, Runtime, Accessibility, Performance and Delivery Architecture for Omnira

Version 1.0 · Juli 2026

CANONICAL

FOUNDER APPROVED

Versionsinformation

Fält	Värde
Titel	Omnira Intelligence Graph — Architecture, Motion and Live Operational Experience
Version	1.0
Datum	10 juli 2026
Status	CANONICAL — FOUNDER APPROVED (11 juli 2026)
Underlag	Tio godkända källdokument v1.0 (se Källmanifest)
Omfattning	Redaktionell konsolidering — ingen implementation
Kanonisering	Godkänd av grundaren 11 juli 2026

Dokumentstatus

CANONICAL — FOUNDER APPROVED. Grundaren granskade och godkände den slutliga PDF:en den 11 juli 2026. Denna samlade bok är den kanoniska Intelligence Graph-arkitekturen för Omnira v1.0. De tio godkända källdokumenten kvarstår som proveniens och historiska designunderlag. Framtida materiella arkitekturändringar kräver en ny version och uttryckligt godkännande. Mindre redaktionella korrigeringar kan släppas som en patch-version utan att i tysthet ändra arkitektonisk innebörd.

Upphovsrätt och intern användning

© 2026 Omnira. Med ensamrätt. Detta är ett internt arkitekturdokument avsett endast för internt bruk inom Omnira. Innehållet får inte distribueras, publiceras eller delas externt utan uttryckligt godkännande från grundaren. Dokumentet innehåller inga credentials, secrets eller personuppgifter.

Redaktionell not

Boken är en redaktionell konsolidering av tio godkända arkitekturdokument för Omnira Intelligence Graph. Det redaktionella arbetet har omfattat rubriknormalisering, konsekvent terminologi, korsreferenser, kapitelintroduktioner och kapitelsammanfattningar, konsolidering av upprepade förklaringar samt borttagning av redaktionella artefakter ur källorna (kvarlämnade formulärmarkörer och utgångna "Draft for Review"-huvuden). Inga arkitektoniska beslut har ändrats, försvagats eller lagts till. Detaljerade krav, numeriska budgetvärden, förbudslistor och acceptanskriterier har bevarats i sin helhet. Där källdokumentet upprepade samma krav i flera dokument har boken behållit kravet i dess primära kapitel och ersatt övriga förekomster med korsreferenser. Ett fullständigt källmanifest finns i bokens slut, och en separat redaktionell rapport följer med leveransen.

Innehåll

Sammanfattning för beslutsfattare	1
Inledning	3
Kapitel 1 — Produktvision	5
Kapitel 2 — Motion design	11
Kapitel 3 — Nodernas och relationernas visuella språk	19
Kapitel 4 — Zoom, labels och interaktion	29
Kapitel 5 — Realtime och operativ sanning	39
Kapitel 6 — Tillgänglighet och responsiv upplevelse	48
Kapitel 7 — Performance budget	54
Kapitel 8 — Implementationsfaser	61
Kapitel 9 — Testplan	67
Kapitel 10 — Git- och leveransplan	73
Ordlista	78
Beroenden mellan kapitlen	80
Implementeringssekvens	81
Slutlig acceptans och styrning	82
Källmanifest	84

Sammanfattning för beslutsfattare

Omnira Intelligence Graph är det visuella nervsystemet för hela Omnira: ett operativt gränssnitt som förenar systemarkitektur, operativ övervakning och visuell intelligens i en och samma graf. Den här boken samlar de tio godkända arkitekturdokumenterna för Intelligence Graph i ett sammanhängande verk — från produktvision till leveransplan.

Grafen är inte en dekorativ visualisering. Den är ett operativt gränssnitt för att förstå, övervaka och styra Omnira.

Vad arkitekturen fastslår

Produkten består av tre huvudlägen som delar samma designspråk men löser olika uppgifter.

System Map visar hur Omnira är uppbyggt: projekt, agents, workflows, tools och deras relationer. **Live Operations** visar vad Omnira gör just nu: aktiva runs, väntande approvals, öppna failures. **Execution Replay** ska senare kunna återberätta vad som faktiskt hände under en execution — men full Replay ingår inte i den nuvarande implementationsomfattningen.

Den bärande designprincipen genom samtliga kapitel är *operational truth before spectacle*: ingen animation får representera aktivitet utan verkligt stöd i runtime-data. Grafen får imponera, men den får aldrig animera falsk aktivitet, blanda projektdata eller låta historiska events se live ut. När systemet inte vet ska det kommunicera osäkerhet genom definierade tillstånd som stale och unknown.

Arkitekturen vilar på ett antal invarianter som ingen implementation, inget tema och ingen optimering får försvaga:

- Projektisolering är backend-enforced. Varje projekt är en separat operativ domän och klienten får aldrig ta emot andra projekts privata runtime-data.
- Failures och incidents är skilda begrepp: ett failure är ett immutabelt operativt faktum, en incident är ett hanteringsobjekt med egen livscykel.
- Approvals har en egen livscykel och får aldrig utföras genom ett oavsiktligt klick. Backend verifierar authority; UI:t blir definitivt först efter serverbekräftelse.
- Tillgänglighet är ett parallellt gränssnitt: en synkroniserad listvy, tangentbordsnavigering, screen reader-stöd och reduced motion bär samma information som canvasen.
- Prestanda degraderar presentation före information: glow, partiklar och ambient motion offras före failures, approvals och status.

Vad som ska byggas och hur

Implementationen delas i sex kontrollerade faser — från audit av befintliga kontrakt, via visuell grund, zoom- och labelmotor, Live Operations truth layer och tillgänglighet, till skalning och hardening. Varje fas har uttryckliga exit gates och stoppregler. Arbetet sker på en egen feature branch, aldrig direkt mot main, och ingen merge sker före godkänd build, testning, säkerhetsgranskning och Codex-review. Den befintliga MVP:n bevaras; ingen total omskrivning görs utan att audit visat objektivt behov.

Bokens status

Samtliga tio källdokument är granskade och godkända av grundaren i version 1.0. Denna samlade bok har status **CANONICAL — FOUNDER APPROVED** och är kanonisk sedan grundaren granskat och godkänt den slutliga PDF:en den 11 juli 2026. Kanoniseringsprocessen beskrivs i bokens avslutande styrningsavsnitt.

Inledning

Bokens syfte

Under arbetet med Omnira Intelligence Graph togs tio styrdokument fram, granskades och godkändes i tur och ordning: produktvision, motion design, visuellt språk, interaktionsstrategi, realtime-strategi, tillgänglighet, performance budget, implementationsfaser, testplan samt Git- och leveransplan. Varje dokument är auktoritativt inom sitt område, men de skrevs som separata leveranser. Den här boken konsoliderar dem till ett sammanhängande verk med gemensam terminologi, konsekvent struktur och korsreferenser — utan att ändra någon godkänd arkitektonisk innebörd.

Boken är skriven för alla som ska implementera, granska eller styra Intelligence Graph: utvecklare, granskare och grundaren själv. Den kan läsas från pärm till pärm, men varje kapitel står också på egna ben.

Hur boken är organiserad

Kapitlen följer källdokumentens godkända ordning, som också speglar arkitekturens logiska beroendekedja:

1. **Kapitel 1 — Produktvision** definierar vad grafen är, dess tre huvudlägen, den visuella hierarkin, projektmodellen och de tio designprinciperna.
2. **Kapitel 2 — Motion design** specificerar hur systemet får röra sig: motion-nivåer, globala tillstånd, Atlas rörelsespråk och nodernas motion states.
3. **Kapitel 3 — Nodernas och relationernas visuella språk** fastställer former, storlekar, färger, statuslager, edge-semantik och design tokens.
4. **Kapitel 4 — Zoom, labels och interaktion** definierar semantic zoom, labelmotorn, selection, drilldown, isolate, search, filter och fullscreen.
5. **Kapitel 5 — Realtime och operativ sanning** låser datakategorier, event-kontrakt, deduplicering, ordering, freshness, freeze och projektisolering i runtime-lagret.
6. **Kapitel 6 — Tillgänglighet och responsiv upplevelse** gör listvy, tangentbord, screen reader, reduced motion och mobilstrategi till förstahandskrav.
7. **Kapitel 7 — Performance budget** sätter mätbara budgetar för frame time, laddning, grafstorlek, labels, motion, minne och nätverk.
8. **Kapitel 8 — Implementationsfaser** delar arbetet i sex verifierbara faser med exit gates och stoppregler.
9. **Kapitel 9 — Testplan** definierar testnivåer, datasets, fasvisa tester, severity-nivåer och release gates.
10. **Kapitel 10 — Git- och leveransplan** styr branch-strategi, commits, migrationspolicy, preview deployment, PR-krav och merge gates.

Efter kapitlen följer en ordlista, en beroendeöversikt mellan kapitlen, en sammanfattning av implementeringssekvensen, ett avslutande avsnitt om acceptans och styrning samt ett källmanifest.

Redaktionella konventioner

Boken är skriven på svenska men behåller Omniras etablerade engelska produkt- och tekniktermer: Atlas, Manager, System Map, Live Operations, Execution Replay, project hub, workflow, agent, run, tool, output, approval, failure, incident, snapshot, realtime, polling, stale, unknown, drilldown, isolate, semantic zoom, reduced motion och Graphify-artifact. Ordlistan definierar samtliga.

Markerade rutor används för arkitektoniska huvudprinciper och invarianter. Numeriska värden som anges som *preliminära* i källdokumentet förblir preliminära: de är budgetvärden som ska verifieras mot den faktiska renderingsstacken och verklig profilering innan de låses.

Källdokumentet bär interna rubriker som "Draft for Review". Dessa var dokumentspecifika arbetsmarkeringar; samtliga tio dokument är godkända i v1.0. Den samlade boken har däremot egen status CANONICAL — FOUNDER APPROVED sedan grundaren godkänt den slutliga PDF:en den 11 juli 2026.

Kapitel 1 — Produktvision

Detta kapitel bygger på Product Vision v1.0. Det definierar vad Intelligence Graph är, vilka upplevelser den ska ge, produktens tre huvudlägen, det visuella grundkonceptet, hierarkin, projektmodellen och de tio designprinciper som resten av boken vilar på.

1.1 Produktvision

Omnira Intelligence Graph ska vara det visuella nervsystemet för hela Omnira. När grafen öppnas ska användaren direkt känna: *hela mitt AI-operativsystem lever, och jag förstår vad det gör.*

Grafen ska förena tre saker som normalt är separata: systemarkitektur, operativ övervakning och visuell intelligens. Den ska kunna visa både hur Omnira är uppbyggt och vad Atlas, Manager, agents och workflows faktiskt gör i realtid.

Den får vara visuellt imponerande, men aldrig på bekostnad av korrekt data, läsbarhet, prestanda, felsökning, tillgänglighet eller projektisolering.

Grafen är inte en dekorativ visualisering. Den är ett operativt gränssnitt för att förstå, övervaka och styra Omnira.

1.2 Den centrala upplevelsen

Intelligence Graph ska ge användaren fem tydliga upplevelser.

Systemet känns levande

Omnira ska upplevas som ett aktivt, intelligent system snarare än en samling statiska tabeller och dashboards. Det åstadkoms genom lågmäld andning i centrala noder, verkliga runtime-signalerna, kontrollerad edge-aktivitet, mjuka tillståndsovergångar, nya noder som växer fram från sin parent samt tydlig respons när Atlas delegerar eller ett workflow aktiveras. Den levande känslan ska vara kontrollerad — en stabil graf får aldrig se stressad ut utan anledning.

Användaren ser vad som händer nu

Live Operations ska snabbt besvara: Vad körs? Vem arbetar? Vad väntar? Vad har misslyckats? Vad behöver mitt godkännande? Vad har precis slutförts? Vilken execution eller agent orsakar en flaskhals? Användaren ska inte behöva tolka en massa rörliga prickar för att förstå läget.

Atlas känns som en verklig central intelligens

Atlas ska vara den största och mest visuellt unika noden — inte bara en ikon med ett namn. Noden ska kännas som en aktiv intelligens som observerar projekten, resonerar, delegerar, tar emot signaler, kommunicerar med Manager och följer pågående operationer. Atlas ska kunna vara lugn och levande utan att fejka pågående arbete. När Atlas faktiskt arbetar ska dess visuella tillstånd förändras utifrån verkliga events.

Problem ska synas omedelbart

Failures, timed-out executions och väntande approvals ska bryta igenom den normala visuella hierarkin. Atlas är störst, men ett kritiskt fel ska ändå kunna dra blicken först — genom varningsfärg, statusring, puls, kontrast, tydlig label och aktivitet i berörd path.

En större nod är strukturellt viktigare. En starkare signal är operativt viktigare. De två systemen får inte blandas ihop.

Grafen ska kunna imponera utan att förlora trovärdighet

Grafen ska ha visuell wow-känsla och kunna demonstrera Omniras kapacitet. Men den får aldrig animera falsk aktivitet, påstå att ett workflow körs när det inte gör det, använda slumpmässiga partiklar som ser ut som runtime-events, visa relationer som inte finns, blanda projektdata eller låta en gammal historisk execution se live ut. Det ska alltid gå att förstå skillnaden mellan visuell presentation, aktuell aktivitet, historisk data och simulerad replay.

1.3 Produktens tre huvudlägen

Intelligence Graph ska inte vara en enda graf med tre filter. De tre lägena ska vara olika visuella produkter som delar samma designspråk.

System Map

System Map visar hur Omnira är uppbyggt. Den svarar på: vilka projekt finns, hur projekten är organiserade, vilka agents och workflows som hör till varje projekt, hur Atlas och Manager är kopplade till systemet, vilka tools, outputs och beroenden som finns, och hur arkitekturen hänger samman.

System Map ska kännas stabil, genomtänkt, arkitektonisk, intelligent, rymlig och nästan stilla. Rörelse används främst för initial stabilisering, filter, drilldown, selection, fokus och ny arkitekturdata. Runtime-status ska inte dominera System Map.

Atlas ligger centralt i den globala kartan och Manager nära Atlas. Varje projekt får ett separat visuellt territorium med egen project hub, egna agents, workflows, tools, outputs och interna relationer. Atlas kan se över projekten, men projektens interna områden får inte visuellt flyta ihop. System Map ska kommunicera: *Atlas har global överblick. Projekten är separata operativa domäner.*

Live Operations

Live Operations visar vad Omnira gör just nu. Den svarar på: vilka workflows som kör, vilka agents som är aktiva, vilka runs som väntar, var failures finns, vilka approvals som blockerar arbetet, vilka outputs som produceras och hur arbetet flödar genom systemet.

Live Operations ska kännas levande, operativ, responsiv, dramatisk när det behövs och lugn när systemet är lugnt — mer som ett nervsystem än ett arkitekturdiagram. Den ska vara visuellt varmare och mer aktiv än System Map.

Som standard visar Live Operations aktiva workflows, aktiva agents, pågående runs, väntande approvals, öppna failures, timed-out executions och nyligen slutförda relevanta runs. Den visar inte automatiskt hela systemets historik; historiska och högvolymsdata aggregeras.

När systemet arbetar ska användaren se riktningen i arbetet, principiellt: Atlas → Manager → workflow → agent → tool → output. Den exakta kedjan varierar beroende på det verkliga runtime-kontraktet.

Execution Replay

Execution Replay ska senare förklara vad som faktiskt hände under en execution eller incident. Replay ska inte försöka visa hela systemkartan utan bygga en förenklad scen med endast relevanta noder och relationer.

Replay ska kunna visa: vad som startade executionen, vad Atlas eller systemet beslutade, vad som delegerades, vilka agents som aktiverades, vilka tools som användes, var väntan eller fel uppstod, hur approval eller retry hanterades, vilket output som producerades och hur executionen avslutades. Replay ska kännas mer som en tydlig berättelse än som en rå loggviewer. All playback ska tydligt märkas som historisk.

Full Execution Replay ingår inte i den nuvarande implementationsomfattningen (se kapitel 8).

1.4 Visuellt grundkoncept

Omniras levande nervsystem

Den övergripande visuella metaforen är ett futuristiskt, biologiskt inspirerat intelligenssystem. Grafen kombinerar neurala nätverk, executive command center, rymdliknande djup, systemarkitektur och kontrollerad organisk rörelse.

Den får inte bli en bokstavlig hjärna, en science fiction-skärmläckare, ett ständigt exploderande partikelsystem, en generisk force graph eller en kopia av Graphify. Målet är att uppnå Graphifys wow-känsla men skapa en tydligt egen Omnira-identitet.

Canvas

Standardcanvasen ska vara nästan svart, djup och rymdliknande, mycket lågmålt texturerad, fri från visuellt brus och kompatibel med både dark och light mode. Dark mode blir den primära cinematic-upplevelsen. Light mode ska inte bara invertera färgerna utan använda ljus kall bakgrund, tunnare shadows, mindre aggressiv glow, tydligare outlines och bibehållit visuellt djup.

Glow

Glow används för att kommunicera identitet, fokus, status, aktivitet och djup — men får inte användas likadant överallt. För mycket glow gör labels svårare att läsa, statusfärger mindre tydliga, grafen visuellt mjuk och grötig och failures mindre uppmärksamhetskrävande. Majoriteten av grafen ska därför vara lågmäld. Stark glow reserveras för Atlas, selected node, aktiv execution, waiting approval, failure och nytt viktigt event.

1.5 Visuell hierarki

Permanent strukturell hierarki

Den permanenta hierarkin avgörs av storlek, form och position:

- **Atlas** — störst, central, unik form och animation, dubbel halo, intern kärna, högsta z-depth.
- **Projects** — näst största nav, tydligt separerade, egna visuella territorier, permanenta labels vid relevant zoom.
- **Manager** — stor executive-nod nära Atlas, mindre än Atlas, visuellt tydligare än vanliga agents, egen form.
- **Workflows** — medelstora strukturella noder som fungerar som operativa nav inom projekt.
- **Agents** — medelstora exekverande noder med tydlig projekt- och funktionskoppling.
- **Runs** — mindre temporära noder som grupperas vid hög volym.
- **Tools** — små funktionella noder som visas tydligare vid workflow- eller run-drilldown.
- **Approvals** — mindre än projekt och Atlas, men med mycket hög attention priority när de väntar.
- **Errors** — kan representeras som status på en existerande nod eller en separat incidentnod; får stark visuell uppmärksamhet utan att bli störst.
- **Outputs** — små eller aggregerade i huvudgrafen, tydligare vid fokus eller drilldown.

Temporär operativ hierarki

Följande kan tillfälligt fånga användarens blick före Atlas: kritiskt failure, waiting approval, timed-out execution, pågående selected execution, retrying execution, nytt viktigt runtime-event, aktivt workflow — och därefter Atlas normala närvaro. Detta sker genom signalering, inte genom att noder permanent ändrar storlek.

1.6 Projektmodell

Varje projekt är en separat operativ domän. Exempel på portföljstruktur: Atlas överblickar projekten The Prompt, Familje-Stunden och GainPilot.

Projekt ska inte dela samma visuella territorium, kopplas direkt till varandras interna agents utan verklig relation, visa varandras runs eller approvals, ärva status från ett annat projekt eller blandas ihop genom en automatisk community-algoritm.

Varje projekt får en egen project hub som fungerar som ankare för agents, workflows, tools, runs, approvals, outputs och incidents.

Filter på ett projekt ska normalt lyfta fram det valda projektet, centrera det mjukt, tona ned övriga projekt, behålla Atlas synligt och bevara den globala mentala kartan. Övriga projekt ska inte försvinna automatiskt; ett separat isolate-kommando kan dölja dem helt (se kapitel 4).

1.7 Community-modell

I **System Map** är den primära communityn projektet, och den sekundära communityn inom projektet är det funktionella området. The Prompt kan till exempel innehålla områdena Intelligence, Content, Publishing, Social, Analytics och Support; Familje-Stunden områdena Content Production, Illustration, Subscription, Customer Support och Analytics.

Communities ska främst vara definierade genom verklig metadata. Automatisk community detection får hjälpa layouten, men ska inte ensam bestämma systemets semantik.

I **Live Operations** är den primära communityn den aktiva executionen eller workflowet, och den sekundära communityn projektet. Live Operations organiserar alltså primärt det arbete som händer, medan System Map organiserar systemets permanenta struktur.

1.8 Stabilitet och rörelse

Initial laddning

När grafen öppnas visas canvas och primära nav först; Atlas etableras först, project hubs växer fram, communities expanderar från sina parents, relationer tonas in, layouten stabiliseras inom cirka 2–3 sekunder och simulationen går därefter ned till vila. Det ska kännas som att systemet organiserar sig, inte som att alla noder kastas runt slumpmässigt.

Viloläge

Efter stabilisering ska majoriteten av noderna vara nästan stilla. Tillåtna viloeffekter: mycket svag andning, subtil halo-variation, långsam intern Atlas-animation, nästan omärkbar parallax eller depth samt lågmäld selected-state-animation. Vanliga noder ska inte flyta omkring kontinuerligt.

Dragning

Användaren kan dra noder för att undersöka strukturen. När sidan laddas om återställs den deterministiska layouten. Permanent pinning kan utvärderas senare men ingår inte i den första visual motion-fasen.

Paus och frysning

Två separata kontroller ska finnas:

- **Pause motion** stoppar force simulation, dekorativ andning, edge-partiklar och pulserande visuella effekter — men stoppar inte datauppdateringar.
- **Freeze live view** stoppar att nya runtime-events visuellt förändrar den inspekterade grafen. Nya events kan fortfarande samlas i en kö eller activity rail. Användaren kan sedan återuppta live-läget, hoppa till senaste event eller behålla det frusna ögonblicket under inspektionen.

1.9 Produktens designprinciper

Princip 1 — Operational truth before spectacle. Ingen animation får representera aktivitet utan verkligt stöd i runtime-data.

Princip 2 — Quiet by default, expressive when necessary. Systemet är lugnt när inget viktigt händer.

Princip 3 — Atlas is the center, not the only signal. Atlas är grafens centrum, men failures och approvals kan tillfälligt ta uppmärksamheten.

Princip 4 — Progressive disclosure. Grafen visar inte allt samtidigt. Mer information visas genom zoom, focus, hover, selection, drilldown och inspector.

Princip 5 — Stable mental map. Filter och uppdateringar ska inte kasta om grafen mer än nödvändigt.

Princip 6 — Projects remain isolated. Grafens visuella struktur måste följa den verkliga projektisoleringen.

Princip 7 — History remains history. Ett resolved failure får inte skrivas om till att aldrig ha hänt.

Princip 8 — Accessibility is a parallel interface. Grafen kompletteras av list- och tabellrepresentationer. Canvasen får inte vara den enda vägen till informationen.

Princip 9 — Scale through aggregation. Stora datamängder hanteras genom clustering, summaries och drilldown, inte genom att rendera allt samtidigt.

Princip 10 — Motion must explain state. Rörelse ska förklara stabilisering, aktivitet, förändring, fokus, status eller riktning. Om en animation inte förklarar något bör den tas bort.

1.10 Icke-mål

Den här fasen ska inte: bygga ett generellt visualiseringsverktyg, visa alla historiska runs samtidigt, göra alla noder konstant rörliga, skapa falska demosignaler, bygga full Execution Replay, göra mobilen identisk med desktop, ersätta vanliga dashboards och loggar, låta användaren ändra systemets runtime-status direkt i grafen utan korrekt backend-action, bygga permanent manuell layoutredigering eller ändra projektisoleringens säkerhetsmodell.

1.11 Framgångskriterier

Produktvisionen är lyckad när:

- Atlas känns som Omniras levande centrala intelligens
- projekt går att skilja åt direkt
- aktiva workflows syns utan att allt annat blinkar
- waiting approvals är omöjliga att missa
- failures upptäcks inom några sekunder
- labels överlappar mycket mindre än i nuvarande graf
- användaren kan förstå en execution path
- grafen förblir användbar efter den första wow-effekten
- användaren förstår skillnaden mellan live och historisk data
- normal användning inte orsakar hög konstant CPU-belastning
- reduced motion ger samma information utan beroende av animation
- mobilen prioriterar failures och approvals framför full grafkomplexitet

Kapitelsammanfattning

Produktvisionen definierar Intelligence Graph som ett operativt gränssnitt med tre lägen — System Map, Live Operations och framtida Execution Replay — byggt kring Atlas som central intelligens, strikt projektisolering och principen att operativ sanning alltid går före visuell effekt. Kapitel 2 översätter visionen till ett exakt rörelsespråk.

Kapitel 2 — Motion design

Detta kapitel bygger på Motion Design Specification v1.0. Det definierar de tre motion-nivåerna, grafens globala tillstånd, rörelseintensitet, Atlas rörelsespråk, nodernas och edgarnas motion states, event arrival, activity rail, fokusövergångar samt prestandaregler och förbjuden rörelse.

2.1 Syfte

Motion-systemet ska få Omnira att kännas levande utan att skapa falsk aktivitet, visuell oro eller onödig prestandabelastning. Varje animation måste ha minst ett tydligt syfte: visa att systemet laddar, förklara en förändring, kommunicera ett tillstånd, visa riktningen i en verklig execution, hjälpa användaren att fokusera, bekräfta en interaktion eller markera att mänsklig åtgärd krävs. Dekorativ rörelse får endast användas lågmålt för identitet och djup.

2.2 Tre nivåer av rörelse

All rörelse delas upp i tre nivåer:

- **Presentation motion** — UI-rörelse som inte påstår att något operativt händer: initial grafstabilisering, filterövergångar, zoom och pan, node focus, hover, inspector, fullscreen, community expansion och mjuk ompositionering. Denna rörelse får användas även när systemet är helt inaktivt.
- **Ambient motion** — lågmäld rörelse som får systemet att kännas levande utan att betyda att arbete pågår: Atlas långsamma andning, subtil halo-variation, mycket svag depth/parallax och långsam förändring i community-atmosfär. Ambient motion får aldrig likna aktiva dataflöden, execution-partiklar, varningar eller statusövergångar.
- **Operational motion** — rörelse som representerar verklig aktivitet: execution längs en edge, agent som arbetar, workflow som körs, approval som blockerar en execution, retry, output som produceras, failure, timeout och nya runtime-händelser. Operational motion kräver stöd i verklig runtime-data.

2.3 Globala motion-tillstånd

Grafen ska alltid befinna sig i ett definierat globalt tillstånd.

Loading

Används medan grafkontrakt och nödvändiga data hämtas. Canvas visas direkt, Atlas-position reserveras och en diskret loading-indikator visas nära grafens centrum. Inga fejkade noder får pulsera och inga simulerade execution-edges får visas. Befintlig cache kan visas med tydlig stale-status om produkten senare stödjer det.

Tidsgränser: under cirka 300 ms behövs ingen separat loading-animation; mellan 300 ms och 2 sekunder används ett diskret loading state; över 2 sekunder visas tydlig textstatus. Vid timeout eller fel övergår loading till error state — inte till oändlig animation.

Entering

Grafen har data men har ännu inte etablerat layouten. Ordningen är: Atlas tonas in, Manager etableras nära Atlas, project hubs växer fram, projektterritorier får subtil atmosfär, primära relationer visas, communities och undernoder expanderar från sina parents, och labels introduceras efter att huvudlayouten blivit stabil. Atlas etableras först eftersom användaren ska uppleva att systemet organiseras kring Omniras centrala intelligens.

Stabilizing

Den kontrollerade force-layouten körs. Måttid för en normal graf är 1,8–3,0 sekunder; mindre grafer kan stabiliseras snabbare och stora grafer kan behöva längre tid, men synlig kraftig rörelse ska inte fortsätta obegränsat.

Krav: noder får inte skjutas över hela canvassen utan kontroll, project hubs ska ha starkare positionsankare än detaljnoder, Atlas ska röra sig minimalt, Manager ska hålla en stabil relation till Atlas, communities ska samlas utan att kollapsa, labels visas gradvis när hastigheten sjunker och användaren ska kunna pausa stabiliseringen. Efter tidsgränsen ska kraften reduceras även om fysisk perfektion inte har uppnåtts.

En graf som ständigt försöker optimera sig är inte stabil.

Stable

Grafens normala viloläge: strukturella noder står stilla, Atlas har mycket lågmäld intern animation, vanliga idle-noder har ingen positionsrörelse, community glow förändras knappt märkbart, normala edges är statiska och operational motion visas endast där verklig aktivitet finns.

Paused

Användaren har valt Pause motion. Följande stoppas: force simulation, ambient breathing, edge-partiklar, upprepade pulser, dekorativ halo-animation och kontinuerlig parallax. Följande behålls: datauppdateringar, statiska statusfärger, statiska badges, inspector, search, filter och keyboard navigation. Ett failure förblir alltså rött även om dess puls stoppas.

Frozen

Användaren har valt Freeze live view — detta är inte samma sak som Paused. När live view är frusen hålls den synliga grafens runtime-state kvar, nya events samlas i en buffert, activity rail kan visa att nya events väntar (till exempel "7 nya events") och användaren kan återgå till live eller hoppa till senaste tillståndet. Ingen viktig data ska tappas bara för att presentationen är frusen. Freeze-buffertens regler definieras i kapitel 5.

2.4 Rörelseintensitet

Systemet använder fyra intensitetsnivåer:

- **Level 0 — Static:** reduced motion, paused motion, statisk fallback och mycket stora grafer på svagare enheter.
- **Level 1 — Ambient:** Atlas andas långsamt, lågmäld selected halo, subtil community depth.

- **Level 2 — Active:** running workflow, pågående agentarbete, aktiva execution-edges, output production.
- **Level 3 — Attention:** waiting approval, failure, timeout, kritisk blocker och nya incidents som kräver mänsklig uppmärksamhet.

Level 3 ska användas sparsamt. Om många noder samtidigt befinner sig där måste grafen aggregera eller prioritera — annars upphör signalen att fungera.

2.5 Atlas rörelsespråk

Atlas får ett eget rörelsespråk som inga andra noder använder fullt ut.

Atlas idle

Atlas idle betyder: Atlas är tillgänglig och systemet är aktivt, men ingen specifik execution påstås pågå. Visuellt: långsam inre kärnrörelse, mycket svag dubbel halo, breathing cycle på ungefär 5–8 sekunder, ingen riktad edge-energi och ingen snabb puls. Atlas ska kännas närvarande, inte rastlös.

Atlas processing

Får endast aktiveras om verklig runtime-data visar att Atlas bearbetar eller deltar i en execution. Kärnan får något högre energi, den inre ringen accelererar måttligt, relevanta utgående eller inkommande relationer aktiveras, berörd execution path lyfts fram och Atlas label kan kompletteras med konkret status — till exempel "Atlas · Analyserar signaler", men inte "Atlas tänker..." om systemet saknar stöd för att påstå detta.

Atlas delegation

När Atlas faktiskt delegerar lämnar en riktad energisignal Atlas och rör sig mot Manager, workflow eller annan verklig target. Edgen får tydlig riktning och målnoden reagerar först när delegationen når den. Animationen spelas inte i loop efter att eventet är avslutat.

Atlas waiting

Atlas får inte se aktiv ut när den i själva verket väntar. Om Atlas väntar på approval eller extern input saktar den inre rörelsen ned, relevant approval edge blir tydlig, Atlas förblir levande men inte operativt aktiv och den blockerade pathen får waiting-status.

Atlas failure

Atlas-noden ska inte automatiskt bli röd bara för att en underliggande execution misslyckats. I stället markeras berörd path, failure-noden eller den exekverande noden får röd status och Atlas kan få en mindre incidentindikator. Endast ett verkligt Atlas-relaterat fel förändrar Atlas huvudstatus. Detta förhindrar att centrumnoden signalerar systemkollaps vid varje lokalt problem.

2.6 Nodernas motion states

Idle

Statisk position, normal basfärg, låg eller ingen glow, inga partiklar och inga upprepade statuspulser. Agents och workflows ska normalt vara helt stilla i idle.

Running

Cyan eller turkos statusring, långsam kontrollerad ringrotation eller energiförskjutning, aktiva edges med riktad rörelse, stabil nodposition och label om density-reglerna tillåter det. Running betyder arbete, inte fysisk nodrörelse.

Waiting

Waiting utan approval: långsam neutral eller amber statusring, ingen execution-rörelse framåt, eventuell inkommande path är statisk och inspector visar väntansorsaken — till exempel väntar på tool, väntar på extern respons eller väntar på dependency.

Waiting for approval

Detta får hög attention priority. Approval representeras av en egen diamond-nod med amber/guld puls; den blockerade execution-edgen slutar vid approval-noden, som får permanent label medan den är öppen. Pulsen är tydlig men inte snabb eller aggressiv, och en count visas vid flera aggregerade approvals. Approval ska inte använda samma röda språk som failure.

Completed

Övergången består av en kort bekräftelseflash, statusringen sluter sig, eventuell output edge aktiveras och noden tonas ned till completed state. Rekommenderad celebration: 300–600 ms, lågmäld, ingen konfetti, ingen kraftig expansion, och den spelas endast en gång per verklig completion. Completed-noder i Live Operations stannar under sessionen enligt retentionregler, men visuellt nedtonade.

Failed

Tydlig röd statusring, broken ring eller avbruten edge, kort initial failure-impact och därefter långsam varningspuls så länge incidenten är öppen, samt permanent label i relevant viewport. Failure får inte döljas av normala density-regler.

När incidenten kvitteras stoppas den aggressiva pulsen, den röda statusen finns kvar och incidenten markeras acknowledged. När incidenten löses övergår noden till nedtonad resolved-failure, den historiska failure-statusen bevaras och en eventuell korrigerande run kan kopplas visuellt.

Cancelled

Nedtonad grå/röd ring, ingen puls efter den initiala statusövergången, berörd path avbryts. Statusen är tydlig men lägre prioriterad än failure.

Timed out

Orange-röd broken ring, kort expansionspuls, eventuell timer-symbol eller badge; aktiv execution-rörelse stoppas och incidenten stannar synlig tills den hanteras eller filtreras bort. Timed out ska kunna särskiljas från generiskt failure utan att användaren behöver öppna inspector.

Retrying

När runtime-kontraktet stödjer retry ligger det ursprungliga failure kvar historiskt, en ny retry-edge eller ny run-nod skapas och en riktad amber/cyan-animation visar det nya försöket. Retry får inte visuellt skriva över den misslyckade executionen.

Selected

Selected är ett interaktionstillstånd, inte en runtime-status. Noden får en tydlig neutral fokusing, höjs i z-depth, de närmaste relationerna förstärks, övriga noder dimmas måttligt, labeln visas alltid, inspector öppnas och grafen centrerar noden mjukt. Selected-stilen måste kunna samexistera med failure, approval och running.

Hovered

Snabb men lågmäld halo, label visas och pointer target förstärks — inga layoutförändringar. Hover får aldrig starta force simulation.

Filtered och dimmed

En nod som inte matchar filtret men ska ligga kvar får reducerad opacity, dold label, borttagen glow och reducerad edge-kontrast, men behåller sin plats. Skillnaden mellan filtered (noden matchar inte ett aktivt filter) och dimmed (noden ligger utanför aktuellt fokus eller vald path) ska finnas i det interna visuella state-kontraktet även om uttrycken liknar varandra.

2.7 Edge motion

Edges skiljs inte bara genom färg utan även genom riktning, mönster och status. Den fullständiga relationssemantiken definieras i kapitel 3; här definieras rörelsen.

- **Normal relation:** tunn, låg kontrast, statisk, ingen partikel; riktning visas endast där relationen faktiskt är riktad.
- **Active execution:** starkare kontrast och riktad energi- eller partikelsignal — endast under verkligt pågående execution. Hastigheten ska vara lugn nog att följa visuellt och animationen avslutas när executionen avslutas.
- **Delegation:** riktad puls från source till target som spelas vid delegationsevent; därefter kan edgen behålla ett svagt active state om uppdraget fortfarande pågår. Source och target får kort responsanimation.
- **Tool use:** särskilt mönster, exempelvis kortare segmenterad energi, från agent/run till tool; toolet kan reagera med en kort aktiveringsring. Återkommande tool calls batchas visuellt vid hög frekvens.
- **Produced output:** signal från producerande run eller workflow till output, kort expansion när output skapas; output-noden växer fram från sin parent och får sedan statiskt state.
- **Approval request:** execution path leds till approval-noden, edgen blir amber/guld och rörelsen stannar visuellt vid approval. Efter approval fortsätter pathen (approved), avslutas tydligt (rejected), får timeout-liknande status (expired) eller dämpad avslutning (cancelled).
- **Failure:** edgen kan brytas visuellt nära failure point; röd signal får inte fortsätta som om executionen fortfarande kör. Ett kort bakåtriktat eko kan visa var felet uppstod, men får inte loopa permanent.
- **Retry:** ny edge eller ny execution-indikator; den tidigare failure-edgen ligger kvar historiskt och retry får eget försökstillstånd. Flera retries aggregeras efter definierad gräns.

2.8 Event arrival

När ett nytt verkligt event når grafen beror presentationen på om objektet redan finns.

- **Befintlig nod ändrar status:** nodens position förändras inte, statusövergången animeras lokalt, relevanta edges uppdateras, activity rail får ett nytt event och grafen får inte destabiliseras.
- **Ny nod skapas:** noden förankras vid sin verkliga parent, börjar liten och lågmäld, växer fram, kopplas med edge, får sitt verkliga statusläge och deltar i en begränsad lokal layoutjustering. Hela grafen får inte starta om sin simulation för en ny mindre nod.
- **Många events samtidigt:** events batchas i korta intervall, preliminärt 250–750 ms beroende på eventtyp. Kritiska events som failure och approval får bryta igenom batchen snabbare; vanliga progress-events kan samlas.

2.9 Activity rail

Activity rail är en del av motion-strategin eftersom den avlastar grafen. Den ska visa tydlig text för: workflow startade, agent aktiverades, tool användes, approval begärdes, failure inträffade, retry startade, output producerades och run slutfördes.

När användaren klickar på ett event fokuseras relevant nod, execution path framhävs, inspector öppnas och grafen flyttar sig mjukt till objektet. Activity rail får inte visa cross-project events när projektvyn är scoped till ett projekt.

2.10 Fokusövergångar

- **Klick på nod:** inspector öppnas, noden centreras mjukt, de närmaste relationerna förstärks och resten dimmas — ingen dramatisk zoom, och användarens nuvarande zoomnivå respekteras så långt det går. Rekommenderad övergång: 250–450 ms.
- **Drilldown:** vald nod behåller sin position som visuellt ankare, irrelevanta noder tonas ut, relevanta undernoder expanderar, camera fit justeras och breadcrumb uppdateras. Drilldown ska inte upplevas som att användaren kastas till en helt annan slumpmässig graf.
- **Reset:** lämnar inspector om användaren väljer det, återställer filter/fokus enligt tydlig kontroll, återgår till deterministisk baslayout med mjuk övergång och startar inte en lång dramatisk force simulation.

2.11 Reduced motion och accessibility

När operativsystemet eller Omnira har reduced motion aktiverat visas force-layouten direkt i slutposition eller med minimal fade, Atlas breathing stoppas, edge-partiklar ersätts av statiska riktade edges, pulser ersätts av statiska statusringar, celebration-flash tas bort eller görs omedelbar, nya noder visas med kort fade, focus använder omedelbar eller mycket kort övergång och inga upprepade animationer förekommer. Informationsinnehållet måste vara oförändrat.

Keyboard focus ska vara minst lika tydlig som hover: fokusindikatorn får inte endast bero på färg, Enter öppnar inspector eller aktiverar primär handling, Escape lämnar focus/drilldown, relationer ska kunna nås genom definierad navigering och den parallella listvyn ska följa samma selection state.

Status måste även kommuniceras genom form, ringmönster, ikon, label, badge och edge-mönster. Röd och grön får aldrig vara enda skillnaden mellan failure och completed. Den fullständiga tillgänglighetsstrategin definieras i kapitel 6.

2.12 Prestandaregler för motion

Force simulation ska bara köras när den behövs, stoppas efter stabilisering, återstartas lokalt vid mindre förändringar, inte köras kontinuerligt i bakgrunden och kunna flyttas till Web Worker om vald grafteknik stödjer det säkert.

Animationer ska samordnas genom requestAnimationFrame. Undvik separata timers per nod, obegränsade CSS-animationer på hundratals element, layout-thrashing, skuggor och blur-effekter som multipliceras okontrollerat samt partiklar på alla edges.

Kvaliteten kan reduceras dynamiskt genom att minska antal partiklar, stoppa ambient animation, minska glow-radius, förenkla shadows, minska label-frekvens, aggregera runs, begränsa edge-animationer till selected/critical paths och stänga av community-atmosfär. Statusinformation får aldrig tas bort som en prestandaoptimering.

Motion-systemet följer de preliminära grafstorleksnivåerna Small (upp till cirka 150 synliga noder, full motion quality), Medium (cirka 150–500, reducerad ambient motion och tätare clustering), Large (cirka 500–1 500, kraftig aggregation och minimala partiklar) och Extreme (över cirka 1 500 relevanta objekt, summary-representation och progressiv laddning). Nivåerna definieras i detalj i kapitel 7 och är initiala budgetvärden som ska verifieras mot den faktiska renderingsstacken.

2.13 Rörelse som aldrig får förekomma

Följande förbjuds: slumpmässiga partiklar längs inaktiva edges, konstant studsande noder, att alla noder pulserar samtidigt, röd puls för icke-kritiska states, loopad completion-celebration, execution-animation utan runtime-stöd, full grafomlayou för varje nytt event, motion som fortsätter när fliken inte är synlig, animationer som gör labels svårare att läsa, abrupt zoom som orsakar desorientering samt rörelse som saknar reduced motion-fallback.

2.14 Acceptanskriterier

Motion-specifikationen är korrekt implementerad när:

- grafen stabiliseras och sedan tydligt går till vila
- Atlas känns levande utan att fejka arbete
- running kan förstås utan att noden flyttar position
- approval är omöjlig att missa men inte ser ut som failure
- failure förblir tydligt tills incidenten hanterats
- completion bekräftas kort och tonas sedan ned
- endast verkliga executions skapar riktad edge-rörelse
- nya events inte förstör användarens mentala karta
- Pause motion och Freeze live view fungerar som två separata funktioner
- reduced motion ger samma information statistiskt
- CPU/GPU-belastningen sjunker efter stabilisering

- bakgrundsflikar inte fortsätter rendera onödig animation
- grafen förblir läsbar när flera operations pågår
- kritiska events prioriteras utan att hela canvasen blir ett larmsystem

Kapitelsammanfattning

Motion design delar all rörelse i presentation, ambient och operational motion, ger grafen sex globala tillstånd (loading, entering, stabilizing, stable, paused, frozen) och fyra intensitetsnivåer, samt ger Atlas ett eget rörelsespråk som aldrig fejkar arbete. Kapitel 3 fastställer de statiska formerna, färgerna och relationsmönstren som rörelsen appliceras på.

Kapitel 3 — Nodernas och relationernas visuella språk

Detta kapitel bygger på Node & Edge Visual Language v1.0. Det fastställer nodernas anatomi och storlekshierarki, det visuella språket för varje nodtyp, statuslagret, edge-semantiken, färgsystemet, dark och light mode, labels, z-depth samt design token-arkitekturen.

3.1 Syfte

Det visuella språket ska göra det möjligt att förstå grafens struktur och operativa tillstånd innan användaren behöver läsa detaljer i inspector. Användaren ska snabbt kunna se vad som är Atlas, vad som är ett projekt, vad som är Manager, vilka objekt som är agents respektive workflows, var en execution befinner sig, om något körs, väntar, har misslyckats eller kräver approval, vilken riktning en relation har, vilka noder som hör till samma projekt, vilka objekt som är strukturella respektive temporära, och vilken information som är aktiv, historisk eller aggregerad.

Systemet ska kombinera geometrisk tydlighet, organisk intelligens känsla, executive professionalism, hög läsbarhet och framtida utbytbarhet.

3.2 Grundprinciper

Form visar identitet. En nods grundform visar vad objektet är — och formen ändras inte bara för att nodens status ändras.

Formspråket i korthet: cirkulär intelligent kärna för Atlas, stor territoriell hub för projekt, hexagonal executive-nod för Manager, cirkel för agent, rundad hexagon för workflow, liten ring för run, diamond för approval, fyrkant eller teknisk capsule för tool, dokumentlik capsule för output samt triangel eller broken ring för incidentindikator.

Färg visar familj och scope. Grundfärg kommunicerar nodtyp, projekttillhörighet och global kontra projektlokal scope. Status läggs ovanpå grundidentiteten genom ring, outline, halo, badge, edge-mönster eller ikon.

En agent får alltså inte plötsligt bli en helt röd cirkel bara för att en run har misslyckats.

Signal visar operativ prioritet. Failures och approvals får fånga blicken genom högre kontrast, starkare statusring, permanent label, puls och särskilt edge-mönster — men de blir aldrig permanent större än Atlas eller projekten.

Visuell semantik ska vara stabil. Samma form, färg och edge-mönster betyder samma sak i System Map, Live Operations, framtida Execution Replay, listvyn, inspector och activity rail. Vyerna får skilja sig i intensitet och informationsmängd, men inte i betydelse.

Designspråket ska vara tokenbaserat. Färger, storlekar, glow, outlines och edge-mönster styrs genom semantiska design tokens, så att Omniras visuella profil, light/dark mode, high contrast och framtida nodtyper kan förändras utan att grafens datamodell skrivs om.

3.3 Nodens anatomi

Varje fullständig nod kan bestå av upp till sex lager:

1. **Core** — nodens centrala visuella kropp; visar nodtyp, grundidentitet och scope.
2. **Identity outline** — en tunn permanent kontur som bevarar formens identitet även när glow och status läggs ovanpå.
3. **Status ring** — visar running, waiting, approval, completed, failed, timed out med flera; kan bytas utan att core förändras.
4. **Attention halo** — används temporärt för selection, hover, approval, failure, nya viktiga events och Atlas närvaro. De flesta noder ska inte ha stark halo i idle.
5. **Badge** — små indikatorer för antal runs, öppna incidents, approval count, retry count, aggregerade objekt, stale data eller historiskt objekt.
6. **Label anchor** — en definierad position där labelmotorn försöker placera texten. Labeln är inte en del av nodens fysiska storlek och kan flyttas för att undvika kollisioner.

3.4 Storlekshierarki

Storlek anges som relativa visuella nivåer, inte hårdkodade pixelmått; exakta värden justeras mot den faktiska renderingsstacken.

Nivå	Nodtyp	Preliminär relativ diameter
XL	Atlas	1,00
L	Project hub	0,58–0,68
M+	Manager	0,44–0,52
M	Workflow	0,30–0,38
M–	Agent	0,25–0,33
S	Approval och incident	0,20–0,28
XS	Run, tool och output	0,12–0,22

Atlas får aldrig krympa till samma visuella nivå som ett projekt eller workflow. Project hubs är tydliga sekundära nav utan att konkurrera med Atlas. Approval- och incidentnoder är strukturellt små men kan få mycket hög attention priority. Runs, tools och outputs visas främst genom zoom, focus, active state och drilldown.

En aggregerad cluster- eller summary-nod får öka måttligt beroende på count, med en begränsad skala i stil med basstorlek plus logaritmen av antalet. Den får aldrig växa linjärt och bli större än projektets hub.

3.5 Atlas visuella konstruktion

Atlas ska vara en intelligent orb, inte en vanlig cirkel. Den består av en ljus inre kärna, ett halvtransparent energilager, en tunn intern ring, en tydligare yttre identitetsring, en mjuk dubbel halo och en stabil label under eller strax ovanför noden. Atlas ska kännas organisk men tekniskt kontrollerad.

Atlas ska fortfarande gå att identifiera när glow är avstängd, reduced motion används, grafen är utzoomad, skärmen visas i light mode eller färgseendet är begränsat — därför har Atlas en unik flerskiktssstruktur även utan animation.

Atlas färgidentitet är en global signaturpalett: preliminärt kall cyan, elektrisk blå, subtil violett och en vit eller ljus kärna. Atlas använder inte projektens lokala identitetsfärger.

Atlas basidentitet behålls alltid; status läggs ovanpå: processing ger starkare intern energi, delegation en riktad aktiv edge, waiting en långsammare core med extern waiting-markör, incident en liten incidentbadge och selected en neutral eller vit fokusring. Ett lokalt projektfel får inte göra hela Atlas röd.

På låg zoom visas labeln "Atlas". På normal och hög zoom kan labeln kompletteras med verifierad status, till exempel "Atlas · Aktiv" eller "Atlas · Väntar på approval". Statusfrasen måste komma från verkligt runtime-stöd.

3.6 Project hub

Project hubben är en stor, mjukt polygonal eller cirkulär kärna med en diskret segmenterad outline. Den ska kännas som ett eget operativt system, ett territoriums centrum och ett nav som innehåller flera funktioner — tydligt mer strukturell än ett workflow.

Varje projekt kan ha en egen accentfärg, men färgen får inte bära hela identiteten. Hubben visar projektnamn, project mode, eventuellt lifecycle mode och tydlig territoriell koppling.

Projektets territorium visas genom mycket svag färgad atmosfär, lokal density, spatial separation, eventuell tunn yttergräns vid focus och en projektlable nära hubben. Territoriet ska inte vara en stor solid bubbla som täcker grafen — det ska kännas som ett ljusfält, inte som ett rektangulärt dashboardkort.

Projektets övergripande health får visas genom en liten sammanfattande ring eller badge, men bara om backend har ett definierat projekt-health-kontrakt. Ett enskilt failure får inte automatiskt färga hela projektet rött. Projektet kan exempelvis visa "2 open incidents", "1 approval", "3 active workflows" — en sammanfattning, inte ett påhittat hälsobetyg.

3.7 Manager

Manager representeras av en mjukt rundad hexagon eller shield-inspirerad nod som kommunicerar koordinering, styrning, delegation och executive kontroll — utan att se militär eller aggressiv ut.

Manager placeras nära Atlas men inte inuti Atlas. Det ska vara visuellt tydligt att Atlas och Manager samarbetar, att Manager är en separat operativ funktion, att Manager inte är en del av ett enskilt projekt om den är global, och att projektspecifika managers får korrekt scope.

Manager använder en global executive-palett — exempelvis blåviolett, kall indigo och neutral metallisk outline — och skiljer sig från Atlas genom mindre glow och tydligare geometrisk form.

Manager kan visa statusarna active delegation, waiting, blocked, incident och selected. En aktiv Manager får inte automatiskt pulsas bara för att andra delar av systemet arbetar.

3.8 Agent

Agents representeras av cirkulära noder med en enkel men distinkt identitetsring. Cirkeln signalerar en individuell exekverande entitet, flexibilitet och deltagande i flera workflows.

Agentens core kan använda projektaccent, agentkategori, avatar eller symbol vid hög zoom samt initialer när ingen ikon finns. Ikoner måste vara konsekventa och får inte göra grafen till en samling små appikoner. Olika agentroller — research, writing, publishing, support, analytics, technical, review — kan skiljas genom mindre sekundära symboler; grundformen förblir cirkulär.

Running visas genom statusring och aktiva edges, inte genom att agenten börjar flyta eller hoppa. Agentens label visas normalt när agenten är selected, hovered, aktiv eller har ett problem, och när zoomnivån och densityn tillåter det.

3.9 Workflow

Workflows representeras av rundade hexagoner — vilket skiljer dem från agents (cirklar), Manager (större och mer executive), tools (tekniskt fyrkantiga) och approvals (diamonds).

Workflow-noden är ett operativt nav som kan bära antal aktiva runs, run cluster, status, trigger-typ och eventuell schedule-indikator. Om flera runs finns visas en liten count-badge (till exempel "3 active") och clustret expanderas vid focus eller drilldown. Workflownoden ska inte omges av tjugo små run-noder i standardvyn om det försämrar läsbarheten.

Workflowets status ska skiljas från en enskild runs status — exempelvis workflow enabled utan aktiv run, workflow med en aktiv run, workflow med failure, workflow paused eller workflow som väntar på approval. Inspector förklarar skillnaden.

3.10 Run

Runs representeras av små ringar eller orbit-liknande noder. De ska kännas temporära, exekverande, kopplade till ett workflow och mindre permanenta än agents och workflows.

Individuella runs visas när de är aktiva, har misslyckats, väntar på approval, är selected, användaren drillar ned eller antalet är begränsat — annars aggregeras de.

En run cluster visas som en större ring med segmenterad outline, count i centrum och sammanfattad statusdistribution — till exempel "12" med indikatorer för 8 completed, 2 running, 1 failed och 1 waiting. Cluster måste kunna öppnas utan att hela grafen kastas om.

3.11 Tool

Tools representeras av små rundade fyrkanter eller tekniska capsules — något som används snarare än något som resonerar. Tool-typen kan visas genom en enkel ikon: database, repository, API, browser, calendar, email, analytics, publishing, MCP eller storage.

Grafen får aldrig visa fullständiga API-nycklar, credentials, secrets eller personuppgifter som inte behövs. Ett tool kan visa att en credential finns, saknas eller är invalid utan att avslöja värdet.

Tools som används av flera projekt får endast visas globalt om användaren har rätt till den vyn. I projektfokus ska cross-project-traversal inte exponera andra projekts interna relationer.

3.12 Approval

Approval representeras av en tydlig diamond — grafens mest igenkännbara action required-form. Öppen approval använder amber, guld eller varm orange som sekundär accent. Den ska inte vara röd, eftersom approval inte betyder failure.

Approval-noden får diamond core, tydlig statusring, permanent label medan den är öppen, en blockerad edge som slutar vid noden och eventuell risk- eller authority-badge.

Flera approvals i samma workflow kan grupperas när användaren är utzoomad, flera requests gäller samma operationstyp eller grafen annars blir överbelastad. Den aggregerade noden visar count. Kritiska eller högrisk-approvals ska kunna undantas från aggregation.

Efter beslut: vid approved avslutas amber-state, execution path kan fortsätta och approvalen blir historisk och nedtonad; vid rejected avslutas pathen tydligt och approvalen behåller sitt historiska beslut; expired eller cancelled får ett särskilt avslutat statusmönster och får inte se aktiva ut.

3.13 Failure och incident

Failure är ett event — incident är ett hanteringsobjekt. En execution kan ha status failed; en tillhörande incident kan vara open, acknowledged eller resolved. Grafen får inte blanda ihop dessa två.

När en run eller agentrelaterad execution misslyckas får den berörda noden röd broken statusring, tydlig failure badge, permanent label i relevant viewport och avbruten eller markerad edge. Nodens grundform och identitet behålls.

En separat incidentnod kan användas när incidenten påverkar flera executions, kräver aktiv hantering, har egen ägare, har resolution state eller länkar till flera berörda objekt. Incidentnoden kan vara en triangle, hexagonal warning marker eller broken ring med varningssymbol. Den ska inte användas för varje litet run-fel.

Färgspråket: open failure är stark röd med hög kontrast och långsam varningspuls; acknowledged är röd utan aggressiv puls med badge; resolved är dämpad röd eller neutral historisk markering med check- eller resolution-indikator — failure-historiken bevaras. Timed out använder orange-röd med timerbadge och segmenterad broken ring, och ska kunna skiljas från generiskt failure även utan text.

3.14 Output

Outputs representeras av små horisontella capsules eller dokumentliknande noder — producerade artefakter, inte utförare. En liten ikon kan skilja kategorier: article, image, video, report, email, dataset, decision, deployment, file, message.

Outputs visas individuellt när de precis producerats, är selected, är kopplade till en fokuserad execution, har failed delivery eller kräver review. Annars kan de aggregeras per workflow eller outputtyp.

3.15 Statuslagret

Status och nodtyp ska kunna kombineras konsekvent:

Status	Visuellt uttryck
Idle	identitetscore, tunn neutral outline, ingen stark halo
Running	cyan/turkos statusring, riktad edge-aktivitet, eventuell running badge
Waiting	amber eller neutral segmenterad ring, statisk blockerad path, väntansorsak i inspector
Waiting for approval	approval diamond, varm amber/guld, permanent label, blockerad edge
Completed	sluten tunn ring, sval grön eller cyan-grön accent, nedtonad opacity efter bekräftelse
Failed	röd broken ring, failure badge, permanent label, markerad failure point
Cancelled	grå diagonal eller avslutad ring, låg kontrast, ingen upprepad puls
Timed out	orange-röd segmenterad ring, timerbadge, tydligt stoppad execution path
Retrying	dubbel statusindikator, historisk failure kvar, ny aktiv retry-path
Selected	separat vit eller neutral fokusring som inte skriver över statusringen
Hovered	kort halo, större interaction target, label visas
Filtered/dimmed	identiteten behålls; status tonas, men kritiska failures och approvals får inte bli osynliga utan ett uttryckligt filterval

3.16 Edge-anatomi och relationstyper

Varje edge kan bestå av base line, directional cue, relation pattern, operational overlay, status marker samt label eller tooltip. Alla lager behöver inte visas samtidigt.

Relation	Exempel	Utseende
Contains	Project → Workflow	tunn solid linje, låg kontrast, svag eller ingen pil vid uppenbar containment
Reports to	Agent → Manager	solid linje, liten riktad markör, mer strukturell än operativ
Delegates to	Atlas → Manager	solid riktad edge, tydligare arrow cue, operational pulse endast vid verkligt delegationsevent
Executes	Workflow → Run	riktad edge, medium kontrast i Live Operations, låg i System Map
Assigned to	Run → Agent	tunn riktad linje, distinkt från delegation genom annat dash- eller arrow-mönster
Uses	Agent → Tool	segmenterad eller kortstreckad linje, operational overlay under verkligt tool use
Produces	Run → Output	riktad edge, kort output-signal vid skapande, statisk efteråt
Depends	—	streckad linje, tydlig riktning mot dependency, etikett vid focus

Relation	Exempel	Utseende
on		
Blocks	—	tjockare kort segmentering, blocker-symbol, hög uppmärksamhet; röd vid failure, amber vid approval eller väntan
Requires approval	—	amber edge riktad mot approval diamond; execution-overlay stannar vid approval

Failure är normalt ett statuslager på en execution-edge, inte en separat permanent relationstyp: edgen bryts visuellt nära failure point med en kort röd markerad sektion, utan fortsatt active-animation efter felet. Retry visas som ny riktad edge eller parallell attempt-indikator; den tidigare failure-pathen bevaras nedtonad och attempt count kan visas vid focus.

Edge-prioritet och density

När många edges överlappar är prioriteten: selected path, active failure path, approval-blocked path, active execution, hovered relation, project-till-Atlas-relation, strukturell relation och sist inaktiva tool/output-relationer. Lågprioriterade edges kan tonas ned, bundlas, döljas vid låg zoom eller visas på hover eller selection — men de får inte raderas ur datarepresentationen.

Projektövergripande edges

Cross-project-edges ska vara ovanliga och kräva verklig semantik. Tillåtna exempel: Atlas observerar projekt, global Manager koordinerar projekt, en auktoriserad global capability används av flera projekt, ett uttryckligt beroende finns eller ett globalt system påverkar flera projekt. En cross-project-edge får inte skapas bara för att två noder har liknande metadata.

Visuellt ska cross-project-edges ligga bakom projektens interna edges, ha lägre normal kontrast, framhävas vid focus, tydligt visa global scope och aldrig möjliggöra otillåten dataläsning.

3.17 Färgsystem

Färger definieras som semantiska tokens, exempelvis graph-global-atlas, graph-global-manager, graph-project-accent, graph-agent, graph-workflow, graph-run, graph-tool, graph-output, graph-status-running, graph-status-waiting, graph-status-approval, graph-status-completed, graph-status-failed, graph-status-timeout, graph-focus, graph-dimmed, graph-edge-structural och graph-edge-active.

Preliminär statuspalett: running är cyan/turkos, waiting neutral amber, approval guld/varm amber, completed sval grön eller mint, failed tydlig röd, timeout orange-röd, cancelled neutral grå, selected vit eller mycket ljus neutral, historical desaturerad blågrå och stale/unknown violettgrå eller neutral varning. Exakta färgvärden ska testas mot Omniras befintliga design tokens och WCAG-kontrast innan de låses.

Projektagent används främst för territory atmosphere, project hub, sekundär outline och lokala community-markörer. Statusfärger måste alltid vinna över projektagent när operativ information kräver det.

3.18 Dark mode och light mode

Dark mode är huvudläget för cinematic-upplevelsen. Canvas: nästan svart med svag kall ton, mycket lågmäld radial depth, ingen stark stjärnhimmel eller dekorativt brus. Noder: ljusare cores, kontrollerad glow, tydliga outlines, mörka interna lager för djup. Edges: låg normal luminans; aktiva edges får högre kontrast och selected paths ska vara tydliga utan att bli vita laserlinjer. Labels: hög kontrast, mörk transparent bakgrund eller text-halo vid behov — text ska inte drunkna i node glow.

Light mode är en egen visuell tolkning, inte en inversion. Canvas: kall ljusgrå eller mycket svagt blåvit med subtil depth, inga hårda vita ytor runt varje nod. Noder: mörkare outlines, mindre glow, tydligare core-kontrast, sparsamma shadows. Edges: mörkare neutral bas; status använder samma semantiska färger och lågprioriterade edges får inte bli osynliga. Labels: mörk text på ljus halvtransparent bakgrund; selected labels får stark outline eller tydlig focus-container.

3.19 Labels

Labelnivåerna är: **permanent landmark labels** (Atlas, project hubs, öppna failures, waiting approvals, selected node), **context labels** (aktiva workflows, Manager när relevant, running agents, fokuserad community) och **detail labels** (runs, tools, outputs, inaktiva agents, relationslabels) som visas genom högre zoom, hover, selection eller drilldown.

En label kan innehålla namn, nodtyp, kompakt status, count och en kort sekundärrad — till exempel "Publishing Workflow / Running · 3 runs". Labels ska inte bli små inspectorkort.

Kollisionsprioriteten är: selected, keyboard focused, hovered, failure, approval, Atlas, running, project, Manager, workflow, agent, run, tool, output.

Långa namn hanteras med en eller två rader, intelligent trunkering, fullständigt namn i tooltip och inspector samt automatisk label-offset — aldrig överlappande text ovanpå nodens center om det gör noden svårsläst. Labelmotorns fullständiga regler definieras i kapitel 4.

3.20 Z-depth och lagerordning

Renderingsordningen är normalt, nedifrån och upp: canvas atmosphere, project territories, lågprioriterade strukturella edges, normala edges, inaktiva detaljnoder, agents och workflows, project hubs, Atlas och Manager, aktiva execution-edges, approvals och failures, selected path, labels samt interaction overlays och inspector anchors. Selected node får höjas utan att hela layouten ändras.

3.21 Tillgänglighet i det visuella språket

Ingen status får kommuniceras endast med färg; systemet använder också ringmönster, form, ikon, label, badge och edge-stil. Små visuella noder ska ha större osynliga hit areas — särskilt runs, tools, outputs, edge selection och mobil användning. Keyboard focus använder tydlig dubbel outline eller fokusring, label, position i den parallella listvyn och announcement via tillgänglig text.

Varje nod ska kunna representeras strukturellt med namn, typ, projekt, status, relationer, senaste event och tillgängliga actions — canvassen får inte vara enda informationskällan. Alla visuella states måste ha ett fullt begripligt statistiskt uttryck i reduced motion. Se kapitel 6 för den fullständiga strategin.

3.22 Responsiv visning

Desktop får fullt nedspråk, territories, större labels, inspectorpanel, fullscreen och högre grafdensity. Tablet får förenklade territories, reducerade labels, begränsat antal edge-effekter och overlay-inspector. Mobilens standardläge är list-/activity-first med failures och approvals högst, mindre graföversikt, färre noder, endast de viktigaste edgarna, större touch targets och inspector som fullskärmspanel eller bottom sheet. Mobilen ska inte försöka visa hela desktopgrafan i miniatyr.

3.23 Design token-arkitektur

Implementationen ska undvika hårdkodad visualisering direkt i node-komponenterna. Preliminära tokengrupper: `graph.node.size`, `graph.node.shape`, `graph.node.stroke`, `graph.node.fill`, `graph.node.glow`, `graph.node.opacity`; `graph.edge.width`, `graph.edge.pattern`, `graph.edge.opacity`, `graph.edge.direction`, `graph.edge.activity`; samt `graph.status`, `graph.project`, `graph.label`, `graph.focus`, `graph.motion` och `graph.quality`.

Exempel på tokens: `graph.node.size.atlas`, `graph.node.size.project`, `graph.node.size.workflow`, `graph.status.failed.stroke`, `graph.status.approval.glow`, `graph.edge.active.width`, `graph.label.priority.failure`. Nodtyp och status mappas till tokens genom ett centralt visuellt kontrakt.

3.24 Utbytbarhet

Det visuella systemet ska stödja framtida experiment utan att ändra graph contract. Följande ska kunna bytas: Atlas orb-design, nodformer, project territory-stil, glow-intensitet, statuspalett, edge-partiklar, label-typografi och kvalitetsnivåer.

Följande får inte ändras av ett tema: betydelsen av status, projektisolering, runtime-sanning, relationssemantik, accessibility-information och prioriteringsregler för kritiska events.

3.25 Förbjudna visuella mönster

Följande ska inte förekomma: alla nodtyper som likadana cirklar; status uttryckt enbart genom färg; project hubs som enorma solida bakgrundsbubblor; glow på alla noder samtidigt; labels inuti små noder när texten inte får plats; edge-färger utan definierad semantik; en röd Atlas på grund av ett lokalt workflowfel; approval och failure med samma visuella språk; historiska runs som ser live ut; cluster-noder utan count eller förklaring; projektaccent som skriver över kritisk status; cross-project-edges utan verifierad relation; visuella relationer som antyder authority när de bara betyder association; samt ikoner som är så detaljerade att de blir oläsliga i grafskala.

3.26 Acceptanskriterier

Det visuella språket är korrekt implementerat när:

- Atlas känns igen utan label
- project hubs går att skilja från workflows
- Manager känns executive och separat från vanliga agents
- agents, workflows, runs, tools, approvals och outputs har tydligt skilda silhuetter
- running, approval, failure och timeout går att skilja utan att endast läsa färg
- approval drar blick utan att se ut som ett fel
- failure syns omedelbart utan att bli grafens största nod
- selected state kan samexistera med runtime-status
- projektterritorier känns separata utan att grafen delas upp i hårda paneler
- cross-project-relationer är visuellt och säkerhetsmässigt kontrollerade
- labels prioriteras och kolliderar betydligt mindre
- dark mode känns cinematic men professionell
- light mode är fullt användbart och inte en enkel inversion
- reduced motion och high contrast kan uttrycka samma information
- stora run-mängder aggregeras begripligt
- varje synlig edge har en definierad betydelse
- nod- och edgeutseendet kan förändras genom tokens utan att graph contract skrivs om

Kapitelsammanfattning

Det visuella språket ger varje nodtyp en egen silhuett, lägger status som ett separat lager ovanpå en stabil identitet, definierar tio relationstyper med egen semantik och styr allt genom design tokens. Kapitel 4 definierar hur användaren navigerar detta språk genom zoom, labels och interaktion.

Kapitel 4 — Zoom, labels och interaktion

Detta kapitel bygger på Zoom, Label & Interaction Strategy v1.0. Det definierar semantic zoom med fem nivåer, labelstrategin och collision avoidance, hover, selection, tangentbordsnavigering, inspector, drilldown, focus och isolate, search, filter, fullscreen, node dragging, live-uppdateringarnas interaktion, mobilstrategi, listvyn samt interaktionshistorik och URL-state.

4.1 Syfte

Strategin ska säkerställa att Intelligence Graph förblir begriplig när användaren är utzoomad, detaljerad när användaren zoomar in, läsbar även med många noder, stabil när filter och live-events förändrar innehållet, användbar med mus, tangentbord och touch, tillgänglig genom både canvas och parallell listvy samt konsekvent mellan System Map och Live Operations.

Grafen får inte försöka visa all information samtidigt. Zoom, labels och interaktioner ska tillsammans skapa progressive disclosure: användaren börjar med en tydlig överblick och får mer information först när den behövs.

4.2 Grundprinciper

Semantic zoom. Zoom ska inte bara göra samma graf större eller mindre — när zoomnivån förändras ska även informationsnivån förändras. Långt utzoomad visas Atlas, projekt och kritiska signaler; vid normal zoom communities, workflows och aktiva agents; inzoomad visas runs, tools, outputs och relationsdetaljer; drilldown visar en full execution chain eller fokuserad community.

Läslighet före fullständighet. Om två labels inte får plats prioriteras den viktigaste och den andra döljs. Grafen ska inte lösa labelproblem genom att göra all text extremt liten, flytta labels långt från sina noder, visa långa ledarlinjer över hela canvasen, låta labels täcka andra noder eller göra canvasen större än användarens viewport.

Stabil mental karta. Användaren ska kunna lära sig var Atlas, projekten och centrala communities finns. Följande får därför inte orsaka full omräkning av layouten: vanlig zoom, hover, selection, öppning av inspector, ett mindre nytt runtime-event, en nod som ändrar status, ett completed event eller ett enkelt filter som dimmar objekt. Större layoutförändring får bara ske vid tydliga handlingar: drilldown, community focus, isolate project, isolate execution path, byte mellan System Map och Live Operations eller manuell reset.

En handling ska ha ett förutsägbart resultat. Klick väljer; dubbelklick kan vara en genväg till drilldown; Enter aktiverar vald nod; Escape går tillbaka; scroll eller pinch zoomar; drag på tom canvas panorerar; drag på nod flyttar noden tillfälligt; en explicit kontroll återställer layouten. Dubbelklick får aldrig vara den enda vägen till en funktion, eftersom det är sämre för touch och accessibility.

Kritisk information får bryta density-regler. Selected node, keyboard-focused node, öppet failure, waiting approval, kritisk timeout och aktivt sökresultat får visas även om normal label-density säger att de borde döljas. Det finns ändå en övre säkerhetsgräns: om hundratals incidents är öppna måste de aggregeras i stället för att skapa hundratals överlappande labels.

4.3 Zoommodell

Exakta numeriska zoomvärden anpassas efter den valda grafmotorn; strategin definierar semantiska nivåer.

Zoom Level 0 — Portfolio overview

Ger en global överblick över Omnira — det globala command center. Visas: Atlas, Manager när relevant, project hubs, projektterritorier, primära Atlas–projekt-relationer, öppna kritiska failures, waiting approvals samt aggregerade incident- och activity-counts. Döljs eller aggregeras: individuella agents, individuella runs, tools, outputs, relationslabels och interna lågprioriterade edges.

Alltid synliga labels: Atlas, projektnamn, öppna kritiska failures, waiting approvals och selected node. Manager-label visas när Manager är aktiv, selected eller relevant för aktuell path. Användaren ska förstå vilka projekt som finns, var något kräver uppmärksamhet och hur Atlas relaterar till portföljen — utan att se intern detaljstruktur.

Zoom Level 1 — Project and community overview

Visar projektets huvudsakliga struktur: Atlas, Manager, project hubs, funktionella communities, viktigaste workflows, aktiva eller problematiska agents, community summary-noder och strukturella huvudrelationer.

Labels prioriteras för Atlas, valt projekt, communities, aktiva workflows, failures, approvals samt selected och hovered nodes. Inaktiva agents får normalt ingen label. Användaren ska förstå hur ett projekt är organiserat utan att behöva se varje run och tool.

Zoom Level 2 — Operational overview

Visar hur arbetet fördelas mellan workflows, agents och aktiva runs: workflows, agents, aktiva run clusters, individuella kritiska runs, approvals, incidents, viktiga tools, aktuella outputs och mer detaljerade execution-edges.

Labels visas för selected och nearby, aktiva workflows, running agents, approvals, failures, fokuserade run clusters samt projekt och Atlas när de ligger i viewport; density-motorn avgör vilka övriga labels som får plats. Detta blir den normala arbetsnivån i Live Operations.

Zoom Level 3 — Detailed inspection

Visar detaljerna runt en vald nod eller community: individuella runs, tools, outputs, relationstyper, retry attempts, lokala dependencies samt fler badges och sekundära labels. Selected + nearby prioriteras; labels kan innehålla namn, status, count, kompakt typ och en kort sekundärrad. Även här gäller collision avoidance. Användaren kan felsöka och förstå en lokal struktur utan att lämna grafen.

Zoom Level 4 — Execution detail

Detta är inte bara en extrem kamerazoom utan ett fokuserat eller drilldown-baserat läge för en run, en approval, ett failure, en execution chain eller ett incidentförlopp. Visas: relevanta noder i executionens ordning, verktygsanrop, approvals, outputs, retries, failure point samt tids- och

statusmetadata. Irrelevanta delar av grafen dimmas kraftigt eller tas bort ur den fokuserade scenen.

Detta läge ligger nära den framtida Execution Replay-modellen men visar aktuell statisk inspektion, inte automatisk historisk uppspelning.

4.4 Zoomövergångar

Desktop ska stödja scroll för zoom, trackpad pinch, zoom mot muspekaren, mjuk men direkt respons och begränsning mot extrema zoomnivåer. Zoom får inte accelerera så aggressivt att användaren tappar orienteringen.

Mobil och tablet ska stödja pinch zoom, tvåfingerspan, dubbeltryck som frivillig zoomgenväg, tydliga touch targets och skydd mot oavsiktlig noddrawing. Enfingersdrag på tom canvas panorerar; enfingersdrag på nod kräver att noden först är tydligt fångad som interaction target.

Grafen ska inte hårt hoppa mellan zoomnivåer — semantic layers tonas gradvis in och ut inom övergångszoner: community-labels tonas in före community-detalljer, agents före deras sekundära labels, och tools och outputs visas först när deras yta är tillräckligt stor. När semantic detail förändras behålls objektens centrala positioner; noder får inte byta plats bara för att en label blir synlig.

4.5 Labelstrategi

Labelklasserna följer kapitel 3: **landmark labels** (Atlas, project hubs, selected node, kritiska failures, waiting approvals) är alltid eller nästan alltid synliga; **context labels** (Manager, communities, aktiva workflows, running agents, fokuserade clusters) visas när de hjälper förståelsen; **detail labels** (runs, tools, outputs, relationer, inaktiva agents) visas vid närmare zoom eller focus; **temporary labels** (hovered node, keyboard focus, nytt event, sökresultat, statusövergång) visas kort eller situationsstyrt.

En label ska vara kompakt — från minimal ("Atlas") via status ("Publishing Workflow / Running") och count ("Content Production / 3 active runs") till failure ("Render Video / Failed"). Labels ska inte visa långa beskrivningar, fulla identifiers, hela felmeddelanden, kompletta timestamps, tekniska payloads eller personuppgifter — det hör hemma i inspector.

Labelmotorn försöker placera labeln i ordningen: under noden, ovanför, till höger, till vänster, diagonal offset och sist dold med åtkomst via hover, focus eller inspector. Prioriterade labels kan använda en kort ledarlinje om labeln måste flyttas; ledarlinjer ska vara korta, ha låg kontrast, endast användas för viktiga labels och inte korsa stora delar av grafen.

Långa namn hanteras genom maximal bredd, en eller två rader, intelligent trunkering samt fullständigt namn i tooltip och inspector. Tekniska suffix eller hashvärden visas inte i standardlabeln om de inte behövs för att skilja två objekt åt — "Generate Voiceover · b5fcaa..." visas normalt som "Generate Voiceover" med identifieraren i inspector.

4.6 Collision avoidance

Labels får aldrig överlappa varandra i standardläget. När konflikt uppstår avgör prioriteringssystemet vilken label som visas.

Prioritetsordningen är: selected, keyboard focused, hovered, critical failure, waiting approval, aktivt sökresultat, Atlas, running node, project hub, Manager, fokuserad community, workflow, agent, run, tool, output.

Varje viewport får en dynamisk labelbudget baserad på zoomnivå, skärmstorlek, node density, prestandanivå, mobil eller desktop samt antal kritiska objekt — inte ett enkelt fast antal. En stor desktop med glest innehåll kan visa fler labels; en mobil med tät graf visar mycket färre; kritiska labels reserverar en del av budgeten och selected + nearby får prioriterad kapacitet.

Varje label får en beräknad bounding box, och motorn undviker konflikt med andra labels, viktig nodgeometri, inspector-overlay, controls, activity rail och viewport-kanter.

Om antalet failures eller approvals blir så stort att alla permanenta labels inte får plats ska systemet visa de högst prioriterade individuellt, skapa aggregerade summary-noder, visa total count och erbjuda filter eller listvy — aldrig bara låta labels staplas ovanpå varandra.

4.7 Hover

Hover på en nod förstärker nodens halo, visar labeln, förstärker direkt berörda edges och visar en kompakt tooltip efter kort fördröjning — men ändrar inte layouten, startar inte force simulation och öppnar inte inspector automatiskt.

Fördröjningar: visuell highlight omedelbart; tooltip efter ungefär 250–400 ms; djupare hover-information endast vid fortsatt hover. Det minskar flimrande tooltips när pekaren passerar flera noder.

Hover på en edge kan visa relationstyp, source, target, aktuell operativ status och senaste relevanta event. Edge hit area ska vara större än den synliga linjen.

4.8 Selection

När en nod väljs får den selected state, labeln visas, inspector öppnas, direkt berörda relationer förstärks, resten av grafen dimmas måttligt och kameran centrerar noden mjukt vid behov med respekt för aktuell zoom. Valet får inte starta en full layoutomräkning.

Selected state ligger som ett separat lager: en selected failure behåller sin röda broken ring, en selected approval sin amber diamond, en selected running agent sin running-ring — den neutrala fokusringen placeras utanför statusringen.

Deselect sker genom klick på tom canvas, Escape, stängning i inspector, reset eller val av annan nod.

4.9 Tangentbordsnavigering

Grafen ska vara fullt navigerbar utan mus: användaren ska kunna fokusera grafen, gå mellan huvudnoder och relaterade noder, öppna inspector, göra drilldown, gå tillbaka, öppna search, använda filter och växla listvy.

Föreslagen navigering: Tab växlar mellan grafkontroller, canvas och inspector; piltangenter navigerar mellan relevanta noder; Enter väljer eller aktiverar fokuserad nod; Space väljer utan att drilldown öppnas; Escape går tillbaka eller avmarkerar; plus och minus zoomar; 0 gör fit graph; snedstreck öppnar search; F fokuserar selected; I isolerar selected path när funktionen är tillgänglig. Kortkommandon ska visas i en hjälpdialog och får inte kollidera med webbläsarens viktigaste standardfunktioner.

Navigeringsordningen ska vara logisk, inte baserad på intern DOM- eller renderingsordning. Prioriterad ordning: Atlas, project hubs, Manager, öppna failures, approvals, aktiva workflows, agents, runs, tools, outputs. Inom en fokuserad community används relationerna och den visuella närheten för att välja nästa nod.

Listvyn ska kunna representera samma selection state: väljs en nod i listan markeras samma nod i grafen och inspector visar samma information; väljs en nod i grafen fokuseras motsvarande listpost.

4.10 Inspector

Desktop: inspector visas normalt som en högerpanel som kan öppnas och stängas, ändra bredd inom definierade gränser, ligga kvar under selection, döljas i fullscreen och visa aktuell nod, status och actions. Grafen anpassar kamerans användbara yta när panelen öppnas, men noderna gör ingen ny simulation.

Fullscreen: inspector kan togglas; standardläget prioriterar canvas; öppnad inspector visas som panel eller overlay; användaren kan återgå till maximal canvas utan att förlora selection.

Mobil: inspector visas som bottom sheet i kompakt läge, fullskärmspanel för detaljer, med swipe eller tydlig stängningskontroll och sticky critical actions för approvals och incidents.

Gemensamma fält: namn, nodtyp, projekt, status, senaste event, relationer, counts, tidsinformation, relevanta actions och länk till full produktvy. Inspector får visa mer data än labeln men prioriterar det som hjälper användaren att förstå eller agera.

4.11 Drilldown

Drilldown visar mer av en vald nods interna struktur utan att användaren tappar varifrån den kom. Den kan aktiveras genom en explicit knapp i inspector, Enter på redan selected node, dubbelklick som desktopgenväg, dubbeltryck på touch när det inte orsakar konflikt eller val i context menu.

Drilldown per nodtyp:

- **Atlas:** Manager, projekt, aktiva globala delegationer, globalt relevanta approvals och incidents, verifierade cross-project-relationer.

- **Project:** projektets communities, workflows, agents, lokala tools, aktiva incidents och approvals. Övriga projekt dimmas eller döljs beroende på valt läge; Atlas behålls som global ankarnod.
- **Community:** tillhörande workflows, agents, dependencies, aktiva runs och outputs på sammanfattad nivå.
- **Workflow:** trigger, agents, aktiva run clusters, tools, approvals, senaste outputs, failures och retries.
- **Agent:** aktuella assignments, workflows, tools, aktiva runs och senaste relevanta outputs.
- **Run:** hela execution-kedjan — start, workflow, agent, tools, approvals, retries, output och failure point.
- **Approval:** begärande objekt, blockerad execution, risk- eller authority-metadata, beslut, konsekvens och eventuell fortsättande path.
- **Failure eller incident:** ursprungligt failure, berörda noder, failure point, felorsak, acknowledgement, resolution samt retry eller korrigering execution.
- **Output:** producerande run, workflow, agent, destination samt review- eller delivery-status.

Drilldown visar alltid en tydlig breadcrumb — till exempel Global → Familje-Stunden → Content Production → Saga Generator — som används för orientering, snabb återgång, keyboard navigation och screen reader-kontext. Back återställer föregående zoom, kameracenter, selection, focus state och de filter som skapades av drilldown. Användaren ska inte kastas tillbaka till en ny layout.

4.12 Focus community och isolate

Focus community är mildare än isolate: communityn centreras, noderna förstärks, övriga noder dimmas, relevanta labels visas, grafens globala struktur ligger kvar och Atlas och project hub förblir synliga när det är möjligt. Detta är standardrekommendationen för normal utforskning.

Isolate döljer irrelevant information och ska vara en tydlig, uttrycklig handling:

- **Isolate project** visar valt projekt, Atlas, relevanta globala relationer och projektspecifik Manager om sådan finns; andra projekts interna noder och irrelevanta cross-project-edges döljs.
- **Isolate community** visar communityns noder och relevanta beroenden.
- **Isolate path** visar en sammanhängande execution chain eller vald relation — för en run: trigger, workflow, agent, tool calls, approvals, outputs, failures och retries.

En tydlig persistent kontroll ska visa att användaren befinner sig i isolate mode — till exempel "Isolated: Render Video execution" med kontrollerna Exit isolate, Back, Fit path och Open in list.

4.13 Search

Search ska kunna hitta Atlas, projekt, Manager, agents, workflows, runs, tools, approvals, incidents och outputs. Sökningen respekterar användarens åtkomst och projektisoleringen.

Resultat visar namn, typ, projekt, status och eventuell matchad identifierare. När ett resultat väljs navigerar grafen till noden, noden väljs, labeln visas, inspector öppnas och de närmaste relationerna förstärks. Om noden inte är laddad i aktuell graf förklarar systemet att detaljer behöver laddas, relevant community eller projekt laddas och användarens aktuella kontext bevaras där det går.

En kort lokal sökhistorik kan stödjas, men känsliga sökningar ska inte automatiskt exponeras för andra användare eller projekt.

4.14 Filter

Filtertyper: projekt, nodtyp, status, tidsperiod, aktivt/problematiskt, approval, failure, community och relationstyp.

Filter ska normalt dimma objekt i stället för att omedelbart ta bort dem — det bevarar den mentala kartan. Ett separat alternativ kan skilja "Dim non-matches" från "Hide non-matches".

Ett filter får dölja failures och approvals endast om användaren uttryckligen valt ett filter som exkluderar dem, och systemet bör då visa att kritiska objekt är dolda — till exempel "3 open incidents hidden by filters".

Dim-filter förändrar inte layouten. Hide-filter får göra en begränsad lokal ompositionering men ska inte kasta om project hubs eller Atlas.

4.15 Fullscreen

Grafen ska stödja riktig webbläsarfullscreen när plattformen tillåter det — för övervakning, demonstration, djup analys och större execution paths.

I fullscreen ska användaren alltid kunna nå: exit fullscreen, search, filter, pause motion, freeze live view, fit graph, reset, inspector toggle, listvy samt motion-/kvalitetsindikator. Kontroller kan döljas automatiskt efter inaktivitet men ska återkomma vid mus-, touch- eller tangentbordsaktivitet.

Ett framtida presentation mode kan minska kontrollerna men får inte skapa falsk aktivitet; det ingår inte nödvändigtvis i första implementationen.

4.16 Fit, focus och reset

- **Fit graph** anpassar kameran till nuvarande synliga scope utan att ändra filter, layout eller selection, och respekterar inspector och activity rail.
- **Focus node** centrerar selected node och dess närmaste relationer, med mild zoomjustering utan abrupt kamerarörelse.
- **Fit path** anpassar kameran till vald execution path eller community.
- **Reset view** återställer kameracenter, zoom, dimming, temporär node dragging och focus state — men behöver inte automatiskt rensa alla filter.
- **Reset all** återställer filter, search, isolate, selection, drilldown, kamera och temporära positioner. Eftersom detta är mer destruktivt ska kommandot vara tydligt namngivet.

4.17 Node dragging

På desktop kan användaren dra en nod för lokal visuell undersökning: noden följer pekaren, direkta edges följer med, närliggande noder kan reagera lätt och hela grafen får inte destabiliseras. När noden släpps stannar den temporärt på plats; positionen gäller under aktuell session eller view state, reset återställer den deterministiska layouten och sidladdning återställer baslayouten.

På mobil kräver node dragging tydlig interaction capture för att inte kollidera med pan — till exempel kort hold före drag, drag endast i explicit läge eller avstängd drag på mycket små noder. Begränsad mobilinteraktivitet prioriteras framför full desktopparitet.

4.18 Live-uppdateringar och interaktion

Om selected node får ett nytt status-event behålls selection, inspector uppdateras, en lokal statusanimation visas och kameran flyttas inte automatiskt.

När en ny nod skapas utanför viewport rycker grafen inte kameran till noden. I stället visar activity rail eventet, en eventuell edge-indikator pekar diskret mot off-screen-aktiviteten, ett kritiskt event kan visa en viewport-notis och användaren väljer själv om den vill navigera dit.

Ett nytt failure eller en ny approval får tydlig notification, activity rail-event, statisk eller pulserande status i grafen och en action för att fokusera — men får inte automatiskt avbryta användarens pågående inspektion annat än vid särskilt definierade katastroflägen.

När Live Operations är fruset fortsätter zoom, pan och inspection att fungera, sökning i det frusna tillståndet fungerar, nya events buffras, användaren ser event count och "Return to live" återansluter grafen till senaste state.

4.19 Interaktion per läge

System Map prioriterar arkitektur, stabilitet, communities, relationer, dependencies och projektisolering. Standardinteraktioner: select, focus community, drilldown, filter på projekt/typ, isolate project, inspect relation, search samt fit/reset. System Map ska inte domineras av runtime-animationer; om aktuell status visas ska den vara sekundär.

Live Operations prioriterar aktiva workflows, running agents, approvals, failures, incidents, execution paths, activity rail och freeze live view. Standardinteraktioner: select active event, follow execution, isolate run path, inspect approval, acknowledge incident, open resolution workflow, freeze och return to live.

Grafen får inte själv ändra backend-status bara för att användaren klickar på en nod. Actions måste använda definierade, behörighetskontrollerade backend-flöden.

4.20 Mobilstrategi

Mobilen öppnar i en activity- och attention-first-vy med prioriteten: failures, approvals, pågående executions, projektstatus, sök och sist graföversikt. Mobilgrafen visar Atlas, valda project hubs, kritiska noder, aktiva workflows och selected path — inte hela desktopgrafen.

Gestures: pinch zoomar, tvåfingersdrag panorerar, tap väljer, tap igen eller explicit action gör drilldown, långtryck ger context actions när det är lämpligt och swipe på inspector expanderar eller stänger.

Persistenta kontroller: back, search, filter, list/graph-toggle, fit current scope och alerts. Fullscreen får inte vara beroende av hover eller små ikonknappar.

4.21 List- och tabellvy

Listvyn är ett tillgängligt alternativ, ett effektivt sätt att hantera många objekt, ett komplement vid felsökning, mobilens primära detaljvy och en backup om canvasrenderingen degraderas.

Listan kan grupperas efter attention required, projekt, status, nodtyp, workflow eller senaste aktivitet. Graf och lista delar filter, search, selection, project scope och live/frozen state. Alla kritiska actions som finns i grafens inspector ska också vara tillgängliga i listvyn. Den fullständiga specifikationen finns i kapitel 6.

4.22 Interaktionshistorik och URL-state

Under en session bör systemet bevara en begränsad navigeringshistorik — valda noder, drilldowns, isolate states, camera states och filterförändringar — som möjliggör Back, Forward och återgång till tidigare analysläge. Historiken är lokal för den aktuella användarsessionen om inget annat beslut fattas senare.

Viktiga navigeringslägen bör kunna representeras i URL när det är säkert och praktiskt: vy, projekt, selected node, drilldown scope, relevanta filter och isolate mode. Det möjliggör refresh utan total kontextförlust, delning av en intern analysvy samt browser back/forward.

URL får inte innehålla secrets, känsliga payloads, obehöriga identifiers eller privat data som inte behövs.

4.23 Telemetri för användbarhet

För att förbättra grafen bör anonymiserad eller intern produkttelemetri kunna mäta: hur ofta search används, vilka zoomnivåer som används, hur ofta labels kolliderar eller döljs, hur ofta användaren öppnar listvyn, vilka drilldowns som används, hur ofta reset behövs, om användaren lämnar fullscreen snabbt, renderingsprestanda, antal synliga noder och edges samt reduced motion- och kvalitetsnivå. Telemetrin får inte logga känsliga graph payloads eller privata execution-detalljer utan uttryckligt stöd.

4.24 Förbjudna interaktionsmönster

Följande ska inte förekomma: hela grafen omorganiseras vid hover; klick gör både selection och oväntad drilldown; dubbelklick är enda vägen till en viktig funktion; zoom gör alla labels permanent synliga; noddragning sparas permanent utan uttryckligt val; ett filter förstör användarens spatiala karta; nya events rycker kameran från pågående felsökning; inspector triggerar ny simulation; mobilgrafen kräver hover; kritiska actions finns endast som små canvasikoner; failures döljs utan att användaren informeras; search visar noder från projekt användaren inte får se; fullscreen saknar tydlig exit; reset betyder olika saker utan tydlig namngivning; listvy och graf visar olika status för samma objekt; visuell selection likställs med backend-action; en approval godkänns genom ett oavsiktligt klick på noden.

4.25 Acceptanskriterier

Strategin är korrekt implementerad när:

- utzoomad graf visar Atlas, projekt och kritiska signaler utan labelkaos
- zoom visar gradvis mer information i stället för bara större objekt
- selected + nearby labels prioriteras
- labels överlappar betydligt mindre än i nuvarande implementation
- viktiga labels vinner kollisioner enligt definierad prioritet
- användaren kan välja en nod utan att layouten kastas om
- inspector öppnas utan att force simulation startar
- drilldown behåller ett visuellt ankare och tydlig breadcrumb
- isolate mode är tydligt och enkelt att lämna
- search kan fokusera en nod och öppna inspector
- filter dimmar som standard och bevarar den mentala kartan
- kritiska dolda objekt kommuniceras tydligt
- fullscreen har fullständig kontroll och tydlig exit
- grafen kan navigeras med tangentbord
- mobilversionen prioriterar failures och approvals
- listvy och graf delar selection, filter och status
- nya live-events inte flyttar kameran utan användarens val
- freeze live view bevarar inspektionen och buffrar nya events
- stora grafer aggregeras i stället för att rendera allt
- reduced motion ger samma information utan lång eller upprepad rörelse
- browser back eller intern back kan återställa tidigare analyskontext
- ett klick på en approval eller incident aldrig automatiskt utför en säkerhetskritisk backend-action

Kapitelsammanfattning

Interaktionsstrategin bygger på semantic zoom i fem nivåer, en labelmotor med strikt kollisionsprioritet, förutsägbara handlingar och en stabil mental karta. Drilldown, focus och isolate ger djup utan desorientering, och inga säkerhetskritiska actions kan utföras av misstag i canvasen. Kapitel 5 definierar det datalager som avgör vad grafen över huvud taget får påstå.

Kapitel 5 — Realtime och operativ sanning

Detta kapitel bygger på Realtime & Operational Truth Strategy v1.0. Det låser datakategorierna, hybrid realtime-modellen, event-kontraktet, deduplicering och ordering, statusmaskiner, approval- och incident-livscyklar, freshness, connection states, freeze-bufferten, projektisolering, authorization och de förbjudna realtime-mönstren.

5.1 Syfte

Strategin ska säkerställa att Live Operations visar verklig operativ aktivitet på ett korrekt, spårbart och säkert sätt. Grafen ska kunna visa vad som körs, vad som väntar, vad som har slutförts, vad som har misslyckats, vad som kräver approval, vilka agents, workflows och tools som deltar, vilka outputs som produceras och hur aktuella uppgifterna är.

Samtidigt måste den undvika: falsk live-känsla, duplicerade events, felaktig eventordning, blandning mellan projekt, kvarhängande running-status, förlorade events vid reconnect, historiska events som ser aktuella ut och visuell aktivitet utan verifierad datakälla.

5.2 Grundprincip

Operational truth before visual activity. Grafen får endast visualisera operativ aktivitet när den kan kopplas till ett verifierat runtime-tillstånd eller event. Om systemet inte vet ska det kommunicera osäkerhet.

Tillåtna osäkra tillstånd: unknown, stale, reconnecting, partially available och "last confirmed at ...".

Otillåtet beteende: anta att en agent arbetar för att ett workflow är enabled; anta att en run är aktiv för att den saknar completion; fortsätta animera en edge efter att datakällan kopplats ned; visa Atlas som processing utan verifierat Atlas-event; behandla gammal pollingdata som realtime.

5.3 Datakategorier

Grafen skiljer på fyra datakategorier.

Snapshot beskriver det senast kända sammanhängande tillståndet: aktiva workflows, aktuella runs, öppna approvals, öppna incidents, agents och deras senaste bekräftade status samt senaste relevanta outputs. Snapshot används vid första laddning, efter reconnect, efter längre frånvaro, när eventsekvensen verkar ofullständig och som kontroll mot realtime-events. En snapshot är inte automatiskt live — den ska ha generated_at, eventuell source_updated_at, projektscope, datakälla och freshness-status.

Realtime event beskriver en verklig förändring: run queued, run started, agent assigned, tool call started, approval requested, output produced, run completed, run failed, retry started, incident acknowledged. Eventet ska förändra ett befintligt state eller skapa ett nytt.

Polled state används för datakällor som saknar push/realtime. Pollingdata behandlas som verifierad data, men presentationen får inte antyda högre aktualitet än pollingintervallet ger — till exempel "Senast uppdaterad för 18 sekunder sedan".

Historical event används för tidigare runs, resolution history, audit, Execution Replay och senaste aktivitet under valt tidsfönster. Historiska events får aldrig automatiskt skapa aktiv edge-motion.

5.4 Hybrid realtime-modell

Realtime används för run start, run status change, approval requested, approval decision, failure, timeout, retry, output produced, incident lifecycle och relevant agent assignment.

Polling används för återkommande consistency checks, datakällor utan realtime, heartbeat och connection health, sammanfattningar, återhämtning efter avbrott, kontroll av långvariga running states samt drift mellan frontend-state och backend-state.

Snapshot används för initial render, reconnect, project switch, tidsfilter, drilldown till objekt som inte är laddade och återställning efter eventgap.

Realtime och polling är inte konkurrenser. Realtime ger snabbhet; snapshot och polling ger korrekthet och återhämtning.

5.5 Event envelope

Varje operativt event bör följa ett gemensamt kontrakt med fält som: event_id, event_type, schema_version, occurred_at, received_at, source, project_id, project_slug, entity_type, entity_id, parent_entity_id, workflow_id, run_id, agent_id, status_before, status_after, severity, correlation_id, causation_id, sequence och payload_summary.

Alla fält behöver inte finnas för varje event, men följande är normalt obligatoriska: unikt event-ID, eventtyp, timestamp, projekt-ID, entity type, entity-ID, källa och schema version.

Grafen ska inte behöva tolka godtyckliga backend-payloads direkt. Ett normaliseringslager översätter källsystemens data till grafens gemensamma eventmodell.

5.6 Eventidentitet och deduplicering

Samma event kan nå klienten flera gånger — genom reconnect, polling, realtime retry, flera subscriptions, replay från servern eller nätverksproblem. Grafen måste därför deduplicera events.

Primär deduplicering sker genom event_id. Sekundär deduplicering kan använda entity-ID, eventtyp, occurred_at, correlation-ID och sequence. Frontend får inte spela samma completion-flash eller failure-puls flera gånger för samma event.

5.7 Ordering

Events kommer inte alltid fram i rätt ordning. Om run.completed anländer före ett försenat tool.started får grafen inte återställa toolen till running.

Ordering använder, när det finns: servergenererad sequence, entity-version, occurred_at och received_at endast som sista fallback. Nyare bekräftad state vinner över äldre state. Ett sent event kan fortfarande sparas i historiken utan att skriva över det aktuella tillståndet.

5.8 Correlation och causation

För att visa en execution chain behövs mer än tidsstämplar. Events kopplas genom **correlation-ID**, som binder ihop events som hör till samma övergripande execution eller process, och **causation-ID**, som visar vilket event eller beslut som orsakade nästa event — till exempel Atlas delegation → workflow start → agent assignment → tool call → output.

Utan verifierad relation får grafen inte dra slutsatsen att närliggande events hör ihop bara för att de inträffade ungefär samtidigt.

5.9 Runtime-statusar och statusmaskin

Preliminärt ska systemet stödja statusarna queued, running, waiting, waiting_for_approval, completed, failed, cancelled och timed_out. Framtida eller möjliga statusar: retrying, paused, blocked, scheduled, skipped, degraded och unknown.

Innan implementation ska de verkliga kontrakten inventeras (se kapitel 8, Fas 0) för att bekräfta vilka statusar som existerar, vilka tabeller och API-routes som producerar dem, vilka som är persisted, vilka som endast kan härledas och vilka transitions som är giltiga. Grafen får inte skapa nya backend-statusar bara för att visualiseringen vill ha dem.

Varje entity type ska ha tillåtna övergångar. Exempel för en run: queued → running → completed; eller queued → running → waiting_for_approval → running → completed; eller running → failed → retrying → running → completed. Otillåtna eller ovanliga övergångar loggas och hanteras defensivt — completed → running får inte automatiskt accepteras för samma attempt utan nytt run- eller attempt-ID.

5.10 Entity truth

Olika objekt har olika sanning:

- **Workflow:** kan vara enabled, disabled, scheduled eller paused — och samtidigt ha noll, en eller flera aktiva runs samt nyliga failures. Workflowets konfigurationsstatus och run-status får inte blandas ihop.
- **Agent:** kan vara available, assigned, executing, waiting, unavailable eller unknown. Agenten visas endast som executing om ett verkligt assignment- eller runtime-event stödjer det.
- **Run:** den primära execution-instansen; bär det mest konkreta runtime-tillståndet.
- **Tool:** visar normalt ett konkret call eller availability, inte en permanent "active"-status.
- **Output:** skiljer på created, processing, delivered, review required, delivery failed och superseded, om backend har stöd för det.

5.11 Approval-livscykel

Approval ska vara ett eget objekt med den preliminära livscykeln requested → pending → approved / rejected / expired / cancelled.

En approval bör ha: approval-ID, projekt, begärande actor, blockerad execution, begärd action, authority/risk level, requested at, decided at, decided by, decision och eventuell kommentar.

En approval-nod får inte försvinna bara för att den inte längre är pending. Den kan tonas ned och bli historisk.

5.12 Incident-livscykel

Failure och incident är inte samma sak. Ett **failure** är ett immutabelt operativt faktum: den här executionen misslyckades. En **incident** är ett hanteringsobjekt med livscykeln open → acknowledged → resolved, eventuellt open → dismissed om governance tillåter det och betydelsen är tydlig.

En incident bör innehålla: incident-ID, failure event, projekt-ID, severity, affected entities, owner, status, acknowledged at/by, resolved at/by, resolution note, korrigerande run eller retry samt relaterade events.

När en incident löses ändras inte den ursprungliga run-statusen från failed till completed.

5.13 Retry och attempts

Retry ska normalt skapa ett nytt attempt, en ny run eller ett tydligt nytt execution-segment. Den tidigare failure-historiken bevaras. Grafen ska kunna visa exempelvis: Attempt 1 — Failed; Attempt 2 — Timed out; Attempt 3 — Running. Retries får inte skriva över det gamla tillståndet och få historiken att se framgångsrik ut i efterhand.

5.14 Freshness och stale state

Varje relevant datakälla ska kunna bedömas som fresh, aging, stale, unavailable eller reconnecting. Preliminära gränser anpassas per källa — för en snabb runtime-källa exempelvis fresh under 15 sekunder, aging 15–60 sekunder och stale över 60 sekunder; för en långsam extern integration fresh enligt förväntat pollingintervall och stale när flera förväntade uppdateringar saknas. Grafen ska inte använda samma freshness-gräns för alla källor.

När data blir stale stoppas eller reduceras pågående edge-animation, objektets senast bekräftade status kan ligga kvar, en stale-badge visas, timestamp visas i inspector och grafen får inte fortsätta påstå "live" — till exempel "Running / Last confirmed 2 min ago / Connection unavailable". Vid långvarig osäkerhet kan state visuellt övergå till unknown; den historiska senaste statusen förblir tillgänglig.

5.15 Connection state

Live Operations ska ha ett tydligt globalt connection state: connecting, live, degraded, reconnecting, offline eller snapshot only. Statusen ska synas utan att dominera UI:t — till

exempel "Live", "Reconnecting · Showing last confirmed state" eller "Snapshot only · Updated 34 sec ago".

En grön "Live"-indikator får endast visas när connection health faktiskt är verifierad.

5.16 Reconnect, heartbeat och health

Vid avbrott behåller grafen det senast bekräftade tillståndet, operational motion stoppas gradvis, UI visar reconnecting och klienten försöker återansluta. Efter reconnect hämtas en ny snapshot som jämförs med lokalt state och eventbuffer; saknade förändringar appliceras, dubletter ignoreras och grafen återgår till live.

Grafen ska inte spela upp alla missade visuella animationer i snabb följd. Kritiska missade events visas i activity rail och state, men skapar inte ett epileptiskt catch-up-flöde.

Realtime-anslutningen bör ha en verifierbar heartbeat eller motsvarande health-signal som visar anslutningens hälsa, senaste kontakt, eventuell lag och subscription-status. Ett öppet websocket utan färskas events är inte automatiskt bevis på att datakällan fungerar.

5.17 Event batching

Vanliga events batchas i korta intervall, preliminärt 250–750 ms. Progress-events, lågprioriterade tool events, upprepade statusuppdateringar och flera completed events kan batchas. Failure, timeout, approval requested, approval decision, critical incident och connection loss ska bryta igenom snabbare. Batching får inte förändra eventens faktiska ordning eller timestamps.

5.18 Freeze live view och freeze-buffer

När användaren fryser Live Operations fryses grafens synliga state; zoom, pan och inspector fortsätter fungera; inkommande events buffras; activity rail kan visa nya event-count; aktuell frozen timestamp visas och backend och eventmottagning fortsätter — till exempel "Frozen at 10:42:18 / 7 new events". Användaren kan välja Return to live, Review buffered events, Jump to latest eller behålla den frusna vyn medan activity rail granskas.

Bufferten ska ha maxstorlek, maxålder, deduplicering, projektisolering och säker återanslutning. Vid mycket hög eventvolym kan bufferten övergå från individuella events till summaries — till exempel "248 progress events / 3 failures / 2 approvals". Kritiska events får inte försvinna i aggregation.

När användaren återgår till live hämtas en färsk snapshot vid behov, buffrade events jämförs, aktuell state appliceras, grafen uppdateras kontrollerat, kritiska nya events sammanfattas och kameran behåller sin plats om möjligt. Grafen behöver inte animera varje mellanliggande event.

5.19 Projektisolering

Varje snapshot, event och subscription ska ha korrekt projectscope. Projektisolering ska vara backend-enforced, inte bara presentation.

I en projektvy får klienten endast ta emot projektets events, tillåtna globala Atlas-events och verifierade globala relationer som är relevanta för projektet. Klienten ska inte ta emot alla projekt och filtrera bort dem enbart i UI, inte kunna söka andra projekts runtime-data, inte få andra projekts payloads i activity rail och inte skapa cross-project execution paths utan explicit semantik.

Atlas Global View får läsa över projekten enligt användarens rättigheter, men varje objekt behåller projekt-ID, scope, source och authority context.

Atlas kan ha events som är globalt observerande, projektspecifika, portföljrelaterade eller governance-relaterade — ett Atlas-event ska därför ange scope (global respektive project med projekt-ID). Grafen får inte koppla ett globalt Atlas-event till samtliga projekt om eventet bara gäller ett projekt.

5.20 Authorization och payload-minimering

Realtime subscriptions och API-anrop ska kontrollera: authenticated user, tenant/owner, project access, role, operation authority, event visibility och approval authority. Frontend får aldrig avgöra om användaren får läsa en incident, godkänna en approval, markera en incident resolved, se ett tool eller följa en cross-project-edge. UI kan dölja funktioner, men backend måste neka otillåtna actions.

Realtime-events ska inte innehålla mer data än grafen behöver. Undvik fullständiga prompts, hemliga tool inputs, API-nycklar, personuppgifter, kompletta dokument, fulla felstackar och känsliga approval-kommentarer. Eventet kan innehålla en säker summary och en referens till en skyddad detaljrout.

5.21 Unknown och conflicting state

Om grafen får ett objekt men saknar tillräcklig information används **unknown**: neutral eller violettgrå status, ingen operational motion, tydlig tooltip, last known state om sådan finns och möjlighet till retry load. Unknown får inte mappas till idle eller completed.

Om två källor motsäger varandra väljer systemet inte godtyckligt. Strategin är: bestäm source authority, jämför entity-version, jämför sequence, jämför occurred_at, visa degraded/uncertain om konflikten kvarstår, logga konflikten och återhämta genom snapshot. Inspector kan visa "Status verification in progress".

5.22 Source authority

Alla datakällor är inte lika auktoritativa: run-tabellen kan vara source of truth för run-status, approval-tabellen för approval-livscykel, incident-tabellen för resolution, worker heartbeat för worker health, medan activity log är historik men inte aktuell entity state.

Innan implementation ska ett source of truth-register skapas som för varje entity type anger primär källa, sekundär källa, event source, snapshot source, freshness policy och konfliktregel.

5.23 Optimistic actions

Actions som acknowledge incident, resolve incident, approve, reject och retry får inte visuellt bli definitiva innan backend bekräftat dem. UI kan visa "Submitting...". Efter bekräftelse uppdateras state, eventet tas emot eller snapshot synkroniseras och det visuella tillståndet ändras. Vid fel återställs tidigare state, ett tydligt felmeddelande visas och approvalen eller incidenten förblir öppen.

Särskilt en approval får aldrig se godkänd ut bara för att användaren klickade.

5.24 Historik, completed state och event-ålder

Live Operations visar inte obegränsad historik. Preliminärt: aktiva objekt alltid inom aktuellt scope; failures tills incidenten hanteras eller filtret ändras; approvals tills beslut eller filterändring; nyligen completed under sessionen eller ett kort tidsfönster; äldre data genom aggregation eller historikvy; full historik genom Replay eller separat detaljvy. Exakta tidsfönster bör vara konfigurerbara.

När en run slutförs verifieras completion-eventet, en kort celebration spelas en gång, output och relationer uppdateras, objektet tonas ned, completed_at visas och objektet markeras som nyligen completed. Om klienten först ser runen efter att den redan är completed spelas ingen falsk live-animation; animation spelas endast när completion-eventet anländer inom ett definierat aktualitetsfönster.

Grafen ska bedöma eventets ålder: ett event från 1 sekund sedan kan skapa en operational transition; ett event från 30 sekunder sedan efter reconnect uppdaterar state med begränsad eller ingen animation; ett event från 2 timmar sedan presenteras historiskt; ett event från föregående dag får ingen live-animation. Aktualitetsgränsen är konfigurerbar per eventtyp.

5.25 Activity rail-sanning

Varje rad i activity rail ska bygga på ett event eller en verifierad state transition, och visa tydlig handling, actor/entity, projekt, tid, status och länk till relevant nod. Den får inte generera spekulativa texter som "Atlas funderar", "Agenten verkar arbeta" eller "Workflowet kanske väntar" — om status är osäker ska det sägas tydligt.

5.26 Realtime i System Map och Execution Replay

System Map kan visa lågmäld aktuell status, men dess huvudsyfte är arkitektur. Realtime-events ska inte omorganisera System Map, skapa stora animationer, dölja strukturella relationer eller göra vyn lika aktiv som Live Operations. System Map kan visa små statusbadges, open incident count, approval count och active workflow count — men Live Operations är den primära runtime-vyn.

Execution Replay ska använda historiskt verifierade events — inte klientens visuella animationslogg. Replay kräver persisted events, event ordering, correlation, causation, attempts, approval history, incident lifecycle och outputs. Replay är alltså beroende av att denna realtime-strategi också producerar bra historik.

5.27 Observability

Realtime-systemet ska kunna mäta: connection uptime, reconnect count, event lag, duplicate events, out-of-order events, dropped events, snapshot duration, snapshot age, buffer size, reconciliation count, unknown states, conflicting states, events per second och render backlog. Detta behövs både för drift och för att veta om grafen verkligen är live.

5.28 Failure modes och schema versioning

Systemet ska ha definierat beteende för: realtime nere men snapshot fungerar (visa "Snapshot only", polling fortsätter); snapshot nere men realtime fungerar (behåll befintligt state men visa degraded eftersom full reconciliation saknas); båda nere (visa offline eller unavailable och stoppa operational motion); upptäckt eventgap (hämta ny snapshot); för många events (aggregera progress-events men behåll kritiska); okänd schema version (ignorera eller quarantine:a eventet och logga det). UI ska inte krascha.

Event envelope har schema_version. Vid förändringar ska systemet stödja bakåtkompatibla tillägg, migrering av consumers, avvisning av okända inkompatibla versioner och tydlig observability. Grafen ska inte vara hårt kopplad till en enda odokumenterad payloadstruktur.

5.29 Empty live state och prestanda

Om inga aktiva eller problematiska operations finns ska grafen vara lugn och visa "Inga aktiva eller problematiska operations just nu", eventuellt med senaste completed, aktuell connection-status, projektstatus och länk till System Map. Den ska inte skapa ambient edge-rörelse för att sidan annars ser tom ut.

Realtime-uppdateringar ska batchas, dedupliceras, endast uppdatera berörda noder och edges, inte starta full force simulation, inte köra collision pass för hela grafen vid varje event, använda lokala state patches, reducera arbete i bakgrundsflik och ha backpressure vid eventspikar. Grafen behöver inte rendera varje tekniskt progress-event visuellt.

5.30 Säkerhetskritiska actions

Grafen ska kunna öppna actions för approval, acknowledge incident, resolve incident, retry och cancel när användaren är behörig. Men: klick på nod väljer bara noden; actions kräver explicit knapp; riskfyllda actions kan kräva bekräftelse; backend verifierar authority; resultatet bekräftas av servern; ett audit event skapas; optimistic success undviks.

5.31 Förbjudna realtime-mönster

Följande får inte förekomma: animation utan verifierat event; att klienten härleder running från frånvaro av completion; att samma event animeras flera gånger; att äldre event skriver över ny state; projektfiltrering endast i frontend; att reconnect fortsätter visa "Live"; att en stale running-nod fortsätter pulsera; att failure ändras till completed när incidenten resolves; att retry skriver över tidigare attempt; att en approval försvinner utan beslut eller lifecycle-event; att websocket-status används som enda bevis för frisk datakälla; att catch-up spelar hundratals animationer; att

historiska events ser aktuella ut; att en backend-action anses lyckad före serverbekräftelse; att okänd status mappas till idle; att Atlas visas som processing utan Atlas-runtime-stöd.

5.32 Acceptanskriterier

Strategin är korrekt implementerad när:

- första render bygger på en tidsstämplad snapshot
- realtime ansluts efter snapshot
- polling eller reconciliation kontrollerar state
- eventdubletter inte skapar dubbla animationer
- out-of-order events inte återställer gammal status
- varje event har korrekt projektscope
- projektvyer aldrig tar emot andra projekts privata runtime-data
- stale data visas tydligt
- operational motion stoppas när live-status inte kan styrkas
- reconnect hämtar eller verifierar en ny snapshot
- freeze buffrar events utan att tappa dem
- return to live återställer aktuell state utan animationskaos
- approvals har egen livscykel
- incidents har open, acknowledged och resolved utan att failure-historiken skrivs om
- retries bevarar tidigare attempts
- completed events visas som historiska när de är gamla
- Atlas processing visas endast med verifierat runtime-stöd
- osäker status visas som unknown, degraded eller stale
- kritiska events prioriteras framför normal batching
- UI-actions blir definitiva först efter serverbekräftelse
- activity rail bygger på verifierade events
- systemet kan mäta eventlag, reconnects och konflikter
- okända eventversioner inte kraschar grafen
- ingen falsk aktivitet visas när systemet är lugnt eller offline

Kapitelsammanfattning

Realtime-strategin gör operativ sanning till ett kontrakt: fyra datakategorier, ett normaliserat event envelope, deduplicering och ordering, egna livscykler för approvals och incidents, ärliga osäkerhetstillstånd och backend-enforced projektisolering. Kapitel 6 säkerställer att denna sanning når alla användare, oavsett enhet och förmåga.

Kapitel 6 — Tillgänglighet och responsiv upplevelse

Detta kapitel bygger på Accessibility & Responsive Strategy v1.0. Det gör den parallella listvyn, tangentbordsnavigeringen, screen reader-stödet, reduced motion, färgoberoende status, high contrast, interaction targets och enhetsstrategin för desktop, tablet och mobil till förstahandskrav — inte efterhandslösningar.

6.1 Beslutssammanfattning

Kapitlet låser följande: grafen ska kunna användas utan mus; canvasen får aldrig vara enda sättet att nå information; en synkroniserad listvy ska alltid finnas; reduced motion tar bort fysik, partiklar och pulser men behåller all information; status visas med form, ikon, mönster och text — inte bara färg; mobilen prioriterar failures, approvals och aktivitet före hela grafen; Pixel 10 Pro får en begränsad, touchanpassad graf; små noder får större osynliga touch- och klickytor; screen reader ska kunna förstå nod, status, projekt, relationer och actions; om grafmotorn inte fungerar ska listvyn fortfarande vara fullt användbar; säkerhetskritiska actions ska aldrig bero på svåra canvasgester; desktop, tablet och mobil får olika informationsdensitet men samma betydelse.

6.2 Syfte och grundprincip

Intelligence Graph ska vara användbar oavsett om användaren använder mus, tangentbord eller touch, om reduced motion är aktiverat, om användaren har nedsatt syn eller färgseende, om enheten är en stor desktop eller mobil, om grafens avancerade rendering måste degraderas eller om canvasen inte kan användas alls. Tillgänglighet är inte ett senare visuellt tillägg utan en del av grafens datamodell, statusmodell, navigering, interaktion, rendering och testning.

Samma operativa information ska vara tillgänglig med och utan grafens visuella effekter. Animation, glow och position får hjälpa användaren, men får aldrig vara den enda bäraren av betydelse.

Ett failure ska kunna förstås genom röd accent, broken ring, failure-ikon, texten "Failed", listposten och screen reader-status — inte bara genom en röd puls.

6.3 Parallell listvy

Grafen ska ha en fullvärdig list- eller tabellvy som kan visa: attention required, failures, approvals, aktiva workflows, running agents, runs, projects, tools, outputs och incidents.

Graf och lista delar selected node, filter, search, projektscope, live eller frozen state, inspector och URL-state där det är lämpligt. När en nod väljs i grafen markeras motsvarande post i listan; när en post väljs i listan fokuseras motsvarande nod i grafen.

Listvyn är inte en reservlösning. Den är särskilt viktig för mobil, många objekt, felsökning, screen reader, reduced motion, stora datamängder samt långsam eller inkompatibel grafikmiljö.

6.4 Tangentbordsnavigering och keyboard focus

Grundkontrollerna följer kapitel 4: Tab växlar mellan toolbar, canvas, listvy och inspector; piltangenter navigerar mellan relevanta noder; Enter väljer eller öppnar; Space markerar utan drilldown; Escape lämnar drilldown, isolate eller selection; plus och minus zoomar; 0 gör fit graph; snedstreck öppnar search; F fokuserar vald nod; I isolerar vald path när det är tillgängligt. Kortkommandon dokumenteras i en tydlig hjälpvy.

Fokusordningen ska vara semantisk — Atlas, projects, Manager, failures, approvals, aktiva workflows, agents, runs, tools, outputs — och får inte bero på slumpmässig renderingsordning.

Keyboard focus ska vara tydligare än vanlig hover: dubbel outline eller separat fokusring, synlig label, hög z-depth, tydlig motsvarande markering i listan och screen reader-announcement. Keyboard focus måste kunna samexistera med failure, approval, running, selected och timeout — fokusringen får inte skriva över statusringen.

6.5 Screen reader-strategi

Canvasen i sig är inte en tillräcklig tillgänglig struktur. En parallell semantisk representation ska exponera: aktuell vy, projektscope, antal synliga objekt, antal failures, antal approvals, selected node, nodtyp, status, senaste uppdatering, relationer och tillgängliga actions.

Exempel på uppläst innehåll: "Render Video, workflow, The Prompt, failed, one open incident, three related nodes."

Följande får annonseras som live announcements: ny approval, failure, timeout, selected node changed, connection lost, live view frozen och live view resumed. Vanliga progress-events ska inte läsas upp ett efter ett — annars blir produkten oanvändbar.

6.6 Reduced motion

När reduced motion är aktiverat stängs följande av: force-stabilisering med synlig rörelse, Atlas breathing, edge-partiklar, upprepade pulser, celebration-animationer, parallax, längre kameraflygningar och kontinuerlig halo-animation.

Följande behålls: statusringar, ikoner, labels, badges, statiska riktade edges, filter, zoomnivåer, search, inspector, listvy och all operativ information. Nya noder får visas genom omedelbar rendering eller mycket kort fade.

Reduced motion får inte innebära reduced information.

6.7 Färg, kontrast och high contrast

Status får aldrig kommuniceras enbart genom färg:

- **Failure:** röd accent, broken ring, failure-ikon, textstatus.
- **Approval:** amber/guld, diamond-form, approval-ikon, permanent label.
- **Running:** cyan/turkos, komplett eller roterande ring i normalt motionläge, statisk aktiv ring i reduced motion, textstatus.
- **Timeout:** orange-röd, segmenterad ring, timerikon, textstatus.
- **Completed:** mint eller sval grön, sluten ring, check-indikator, textstatus.

Exakta färger ska kontrasttestas mot både dark och light mode.

Ett high contrast-läge bör kunna minska transparens, minska glow, förstärka outlines, öka label-kontrast, göra edge-mönster tydligare, använda tjockare fokusringar och minska atmosfäriska bakgrundseffekter. High contrast ska vara kompatibelt med reduced motion.

6.8 Interaction targets

Den synliga noden kan vara liten, men dess interaktionsyta ska vara större — särskilt för runs, tools, outputs, approvals, edge selection och mobil användning. Touch targets ska normalt eftersträva minst omkring 44 × 44 CSS-pixlar. Det innebär inte att den synliga noden måste vara så stor; den osynliga hit arean kan vara större.

6.9 Desktop, tablet och mobil

Desktop är huvudupplevelsen och får stödja full graf, högre node density, hover, inspectorpanel, activity rail, fullscreen, keyboard shortcuts, drag av noder, focus och isolate samt den parallella listvyn. På MacBook Air M4 ska grafen kunna köra sin normala kvalitetsnivå för små och medelstora grafer; vid större grafer sänks kvaliteten automatiskt.

Tablet får färre permanenta labels, enklare project territories, mindre edge-motion, overlay-inspector, större touch targets, mer begränsad node dragging och tydligare list-/graf-växling. Tablet ska inte försöka efterlikna desktop exakt.

Mobil är attention-first med standardordningen: failures, approvals, running executions, projektstatus, senaste aktivitet, search och sist graföversikt. Mobilgrafen visar främst Atlas, project hubs, kritiska noder, aktiva workflows och vald execution path — inte hela desktopgrafen.

Pixel 10 Pro ska få en touchanpassad kompakt graf med pinch zoom, pan, tap selection, större hit areas, bottom sheet-inspector, fullskärmsdetaljer, list-/graf-toggle och reducerad kvalitet vid hög node density. Mobilen ska inte försöka rendera tusentals noder.

6.10 Mobilgester

Tap väljer nod; tap igen eller en tydlig knapp gör drilldown; pinch zoomar; tvåfingersdrag panorerer; långtryck ger context actions där det är motiverat; swipe på inspector expanderar eller stänger; back gesture går tillbaka i drilldown.

En approval får aldrig godkännas genom tap på noden, dubbeltryck, swipe eller långtryck utan separat bekräftelse.

6.11 Inspector per enhet

Desktop: högerpanel som kan togglas, ändra bredd inom gränser och ligga kvar under navigation. Tablet: overlay-panel som kan täcka en del av canvasen med bibehållen selection. Mobil: bottom sheet för sammanfattning, fullskärm för detaljer, sticky actions, tydlig stängning och inga kritiska actions nära swipe-zoner.

6.12 Responsive density

Grafens informationsnivå anpassas efter viewport, pixel density, node count, edge count, input type, prestandaklass, reduced motion och aktuell zoom. Desktop får fler labels och communities; tablet färre labels och mer aggregation; mobil endast kritiska och fokuserade objekt.

Responsiv design ska minska information genom aggregation, inte genom att göra allt pyttelitet.

6.13 Canvas fallback

Om avancerad rendering inte fungerar faller systemet tillbaka stegvis: minska motion, minska glow och atmosfär, minska edge-effekter, minska labels, visa aggregerad graf, visa statisk graf och sist prioritera listvyn.

Användaren ska fortfarande kunna se failures och approvals, söka, filtrera, öppna inspector, utföra behöriga actions och se connection-status.

6.14 Loading och fel

Tillgängliga textstatusar ska finnas för loading, reconnecting, offline, stale, empty, no active operations, rendering degraded och graph unavailable. Exempel: "Grafvisualiseringen kunde inte startas. Samma information finns i listvyn."

Detta ska inte blandas ihop med ett operativt failure inne i Omnira — UI-fel och operativa incidents måste ha skilda representationer.

6.15 Säkerhetskritiska actions

Approve, reject, acknowledge incident, resolve incident, retry och cancel måste finnas utanför canvasens små interaktionsytor: i inspector, i listvyn och i en särskild approval- eller incidentpanel. De ska kräva explicit knapp, backendkontroll, tydlig pending state, serverbekräftelse och eventuell bekräftelsedialog beroende på risk.

6.16 Responsiv fullscreen och textzoom

Desktop får riktig browser fullscreen med komplett kontrollfält; tablet fullscreen eller nästan fullscreen med overlay-controls. På mobil kan grafen expandera till skärmen, men browser chrome kan begränsa fullskärm, exit ska alltid vara tydlig, list-/graf-toggle finnas kvar och inspector får inte bli otillgänglig. Fullscreen får inte vara ett krav för att grafen ska fungera på mobil.

Webbläsarens textzoom ska stödjas. Vid större text får labels större bounding boxes, fler lågprioriterade labels döljs, inspector får reflow, listvyn fortsätter vara läsbar och kritiska actions förblir synliga. Text får inte kapas bort utan möjlighet att läsa fullständigt innehåll.

6.17 Språk och terminologi

Labels använder korta termer; inspector och listvy får använda mer förklarande text. Tekniska engelska termer kan finnas där Omnira redan använder dem, men betydelsen ska vara konsekvent. Statusnamn får inte variera mellan graf, listvy, inspector och activity rail.

6.18 Prestanda och accessibility

Tillgänglighetslagret ska inte byggas genom tusentals tunga dolda DOM-element som uppdateras varje frame. Strategin använder semantisk representation av relevant scope, virtualiserad listvy, begränsade live region-announcements, uppdatering endast vid meningsfull state-förändring och synkroniserad men inte frame-baserad accessibility state.

6.19 Testkrav

Minst följande ska testas: komplett navigation utan mus; screen reader i huvudflöden; reduced motion; high contrast; färgblindhetssimulering; 200 procents textzoom; mobil touch; Pixel-lik viewport; listvy utan canvas; rendering failure fallback; approval- och incident-actions; focus efter drilldown; focus efter inspector close; focus efter reconnect; keyboard exit från fullscreen och isolate.

6.20 Förbjudna mönster

Följande ska inte förekomma: viktig information endast i hover; status endast genom färg; kritiska actions endast i grafen; mobilfunktioner som kräver hover; touch targets som är lika små som synliga run-noder; animation som inte går att stoppa; reduced motion som tar bort statusinformation; screen reader som läser upp varje runtime-event; full desktopgraf nedskalad till mobil; listvy med annan status än grafen; fokus som försvinner efter interaction; canvasfel som gör produkten oanvändbar; approval genom enkelt nodklick; automatisk kamerarörelse som inte kan stoppas.

6.21 Acceptanskriterier

Strategin är korrekt implementerad när:

- grafens huvudfunktioner kan användas utan mus
- screen reader kan förstå relevant nod och status
- reduced motion tar bort rörelse men inte information
- failure och approval kan skiljas utan färg
- listvyn fungerar som fullvärdigt alternativ
- graf och lista delar selection och status
- mobilen prioriterar failures och approvals
- Pixel 10 Pro får en praktisk touchupplevelse
- små noder har tillräckliga hit areas
- kritiska actions finns i tydliga paneler
- rendering kan degraderas utan att operativ information försvinner
- textzoom fungerar utan att controls försvinner
- focus återställs logiskt efter drilldown och inspector
- offline och graf-fel kommuniceras med text

- inga säkerhetskritiska handlingar utförs av misstag genom canvasinteraktion

Kapitelsammanfattning

Tillgänglighet är ett parallellt gränssnitt: en synkroniserad listvy, semantisk tangentbordsordning, screen reader-representation och reduced motion bär exakt samma operativa sanning som canvasen. Mobilen är attention-first, kritiska actions ligger alltid utanför canvasgester, och om renderingen fallerar är listvyn fullt användbar. Kapitel 7 sätter de mätbara prestandamålen som gör detta hållbart.

Kapitel 7 — Performance budget

Detta kapitel bygger på Performance Budget v1.0. Det definierar målenheter, frame- och latensbudgetar, laddnings- och stabiliseringsmål, grafstorleksnivåer, label-, motion-, glow-, event-, minnes- och nätverksbudgetar, kvalitetsnivåer med automatisk degradering samt testscenarier och verktyg.

7.1 Syfte och grundprincip

Performance budget definierar hur mycket grafen får kosta i renderingstid, CPU, GPU, minne, nätverk, animationer, labels, noder, edges och runtime-events. Budgeten ska säkerställa att Intelligence Graph känns snabb, förblir läsbar, inte värmer upp enheten i onödan, inte tömmer mobilbatteriet, inte blockerar resten av Omnira, kan växa genom aggregation och behåller kritisk information vid degradering.

Målet är inte att rendera flest möjliga objekt. Målet är att visa rätt operativ information med stabil prestanda. Degradera presentation före information.

Om enheten eller grafstorleken inte klarar full visuell kvalitet minskas först partiklar, glow, ambient motion, community atmosphere, detaljlabels och lågprioriterade edges. Systemet får inte först ta bort failures, approvals, connection state, selected path, status, tillgängliga actions eller projektisolering.

7.2 Målenheter

Strategin ska minst verifieras på: **primär desktop** — MacBook Air M4 med modern Chromium-baserad browser, normal Retina-upplösning, både batteri och nätström; **sekundär desktop** — användarens stationära dator med vanlig modern GPU eller integrerad grafik och större skärm; **primär mobil** — Google Pixel 10 Pro med Chrome för Android, touch samt varierande batteri- och temperaturläge. Som generell baseline ska grafen fortfarande vara användbar på en vanlig modern kontorslaptop utan dedikerad GPU.

7.3 Frame budget

På 60 Hz-skärmar är frame-budgeten ungefär 16,7 ms — den totala tiden för JavaScript, layout, collision-beräkningar, rendering, compositing och input handling.

Desktopmål. Small och Medium graph: nära 60 FPS under pan och zoom; accepterad tillfällig nedgång 45–60 FPS; ihållande under 30 FPS är inte godkänt. Large graph: minst stabilt omkring 30–45 FPS, där full kvalitet redan ska vara reducerad och interaktionen fortfarande känns direkt.

Mobilmål. Idealiskt 60 FPS i en liten fokuserad graf; godkänt är stabilt 30–45 FPS; vid ihållande under 30 FPS ska grafen sänka kvalitet eller gå till summary mode. Mobilen ska inte försöka hålla desktopkvalitet genom att överbelasta enheten.

7.4 Input latency

Användaren ska uppleva att grafen reagerar direkt: hover-highlight under cirka 50 ms; nodselection under cirka 100 ms; inspector börjar öppnas under cirka 100 ms; pan och zoom svarar inom nästa render frame; search input har ingen märkbar skrivefördröjning; filterkommandon ger visuell respons under cirka 150 ms. Långsammare dataladdning får visas separat — UI:t får inte kännas låst medan backend hämtas.

7.5 Initial laddning och stabilisering

Preliminära mål efter att route och grund-UI laddats: canvas shell visas inom cirka 300 ms; första meningsfulla graf inom cirka 1 sekund efter att snapshot finns tillgänglig; stabiliserad graf för normal small/medium graph inom cirka 1,8–3 sekunder. Om snapshot tar längre tid ska canvassen visas, loading state vara tydligt, inga fejkade noder eller aktiviteter visas och användaren förstå vad som väntas på.

Force simulation får inte köras obegränsat: small graph normalt under 2 sekunder med full visuell kvalitet; medium graph normalt under 3 sekunder med reducerad ambient motion under stabiliseringen; large graph har en hård tidsgräns där simulationen avslutas eller fryses kontrollerat med aggregation och starkare ankare. Efter stabilisering ska positionsberäkningar avta, simulationen stoppas, CPU-belastningen minska och grafen övergå till stabilt viloläge.

7.6 Grafstorleksnivåer

Nivå	Ungefärlig storlek	Regler
A — Small	upp till 150 synliga noder, upp till cirka 500 synliga edges	full visuell kvalitet, Atlas fulla ambient state, community atmosphere, aktiva execution-effekter, normal labelbudget
B — Medium	151–500 synliga noder, upp till cirka 2 000 relevanta edges	reducerad ambient motion, färre labels, edge-animation främst på aktiva paths, run clustering, färre starka glow-effekter
C — Large	501–1 500 synliga noder, flera tusen möjliga relationer	tydlig aggregation, endast kritiska och valda labels, lågprioriterade edges döljs eller bundlas, community summaries, begränsad force simulation, minimala partiklar, Web Worker utvärderas eller krävs
D — Extreme	över cirka 1 500 relevanta objekt	visas inte som full detaljgraf: project summaries, community summaries, counts, statusdistribution, progressiv laddning, listvy, search och drilldown

10 000 objekt i systemet betyder inte 10 000 samtidigt renderade noder.

Följande ska skiljas åt: totalt antal objekt i databasen, objekt i aktuellt projektscope, objekt i valt tidsfönster, objekt laddade i klienten, objekt renderade i viewport samt individuella objekt kontra clusters. Grafen ska kunna representera mycket stora datamängder utan att ladda eller rendera allt samtidigt.

Runs aggregeras när de inte är aktiva, kritiska, selected, approval-blockerade, nyligen relevanta eller del av en fokuserad execution chain. Äldre runs visas genom counts, statusdistribution,

historiklistor, tidsfilter, Replay och drilldown. Historiska runs får inte ligga kvar som tusentals små noder i huvudgrafan.

7.7 Edge budget

Edges är ofta dyrare och visuellt rörigare än noder. Prioriteten följer kapitel 3: selected path, failure path, approval-blocked path, active execution, hovered relation, Atlas/projektrelation, strukturella relationer, inaktiva tool/output-relationer. Vid hög density kan lågprioriterade edges tonas ned, döljas, visas vid selection, bundlas där semantiken tillåter eller laddas vid drilldown. Alla relationer kan finnas i data utan att alla måste renderas samtidigt.

7.8 Label budget

Labels har en dynamisk budget. Desktop, preliminära normalgränser: låg zoom cirka 8–25 labels; normal zoom cirka 20–60; detailed focus upp till cirka 80–120 konfliktfria labels. Mobil: portfolio cirka 5–12; fokuserad graf cirka 10–30. Detta är övre riktvärden, inte krav på att fylla budgeten.

Kritiska labels prioriteras alltid: selected, failure, approval, Atlas, project och running objekt. Labelmotorn ska inte göra ett fullständigt collision pass varje animation frame.

7.9 Motion-, glow- och blur-budget

Ambient motion är tillåten på Atlas, selected node och ett litet antal globala visuella element — inte individuellt på hundratals noder. Operational motion är tillåten endast på verkligt aktiva paths, med en preliminär samtidighetsbudget på cirka 16–24 tydligt animerade edges på desktop och cirka 4–8 på mobil; ytterligare aktivitet aggregeras eller förenklas, och kritiska paths vinner över vanliga active paths.

Glow är visuellt kraftfullt men GPU-mässigt dyrt: stark glow reserveras för få objekt, glow-radius reduceras vid hög density, stora blur-lager dupliceras inte per nod, project atmosphere använder få stora lager i stället för hundratals individuella shadows, light mode använder mindre glow och mobil mindre än desktop. Hög attention-glow får preliminärt endast användas samtidigt på ett begränsat antal objekt; blir de kritiska objekten många ska de aggregeras.

7.10 Runtime-eventbudget och visuell uppdateringsfrekvens

Grafen måste kunna ta emot fler events än den visuellt visar. Initialt mål: ett stabilt flöde på omkring 20 events per sekund och kortare bursts på omkring 100 events per sekund, utan att en separat animation spelas för varje event. Detta ska verifieras mot verklig backend och eventvolym. Strategin är att deduplicera, batcha, koalescera flera uppdateringar till samma entity, prioritera kritiska events, summera progress-events och applicera lokala state patches.

Grafen skiljer på event ingestion rate, state update rate och visual update rate. Vanliga progress-events kan appliceras visuellt i batcher varje 250–750 ms; failures, timeouts och approvals visas snabbare.

7.11 Main thread, Web Worker och render scheduler

Main thread kan användas för små grafer, input, camera, inspector, enkel rendering och mindre lokala layoutändringar. Web Worker bör utvärderas för force simulation över några hundra noder, community-beräkningar, spatial indexing, tung aggregation, label- eller collision-preprocessing och större graph normalization. Valet baseras på den befintliga grafmotorn och profilering — Worker-komplexitet införs inte utan mätbart behov, men arkitekturen får inte blockera det.

All kontinuerlig visuell animation samordnas genom requestAnimationFrame. Undvik en timer per nod, en CSS-loop per edge, flera osynkroniserade animationssystem, animation i dolda delar av grafen och setInterval-baserad rendering. En central render scheduler ska veta om motion behövs, om fliken är synlig, om grafen är paused, om reduced motion gäller och vilken kvalitetsnivå som används.

När dokumentet inte är synligt upphör requestAnimationFrame-baserad animation, force simulation pausas, hover- och pointer-arbete upphör, ambient motion stoppas och lågprioriterad visuell processing stoppas. Events kan fortsätta tas emot enligt realtime-strategin. När fliken åter blir aktiv verifieras state, eventuell snapshot hämtas, grafen uppdateras kontrollerat och missade animationer spelas inte upp i snabb följd.

7.12 Minnesbudget

Exakt MB-gräns beror på grafmotorn och browsern, men följande krav gäller: ingen kontinuerlig minnesökning i viloläge; borttagna noder och edges ska kunna garbage collectas; subscriptions stängs vid unmount; eventbufferten har maxstorlek och maxålder; textmätningar och ikoner får cacheas med begränsning; byte mellan projekt behåller inte obegränsade gamla grafer.

Långsessionstest: efter minst 30 minuters användning med projektbyte, zoom, drilldown, live-events, freeze/resume och inspector ska minnet stabiliseras och inte fortsätta öka utan gräns.

7.13 Nätverksbudget och caching

Den initiala snapshoten ska minimera payload: summary först, detaljdata vid drilldown, projektscope, tidsfilter, pagination eller cursors där det är relevant och kompakta realtime-events. Undvik hela prompts, dokumentinnehåll, fulla stack traces, alla historiska runs, andra projekts data och detaljer som inte visas. Grafen hämtar djup information när användaren behöver den.

Följande kan cacheas säkert när det är lämpligt: statisk System Map, node metadata, ikoner, textmått, projektfärger, deterministiska positioner och senaste säkra snapshot med timestamp.

Cache får aldrig göra att stale data ser live ut. Alla cacheade snapshots ska visa när de skapades, freshness och connection state.

7.14 Kvalitetsnivåer

- **Quality High** — används när grafen är liten, enheten klarar budgeten, FPS är stabil och reduced motion inte är aktivt. Innehåller full Atlas-animation, community atmosphere, fler labels, fler aktiva edge-effekter och normal glow.
- **Quality Medium** — mindre ambient motion, färre labels, reducerad glow, prioriterade aktiva paths och mer clustering.

- **Quality Low** — ingen ambient motion utöver nödvändig status, inga eller mycket få partiklar, minimalt glow, färre edges, stark aggregation och statiska statusmönster.
- **Quality Static** — reduced motion, mycket stor graf, svag enhet eller rendering fallback. Visar samma operativa sanning utan animation.

Kvaliteten kan sänkas automatiskt när FPS ligger under gränsen under en definierad period, long tasks upprepas, node/edge count överskrider budgeten, mobilen blir termiskt belastad, browsern signalerar minskad prestanda, användaren aktiverar battery saver eller reduced motion används. Systemet ska undvika att snabbt växla fram och tillbaka mellan kvalitetsnivåer; en hysteresis eller cooldown används.

En avancerad användare kan senare få välja manuellt mellan Auto, High, Balanced, Performance och Static, med Auto som standard. Status- och säkerhetsinformation får inte påverkas av valet.

7.15 Long tasks och kameraprestanda

JavaScript-arbete över cirka 50 ms betraktas som en long task och ska mätas. Upprepade long tasks under pan, zoom, eventspikar, collision-uppdateringar, drilldown och projektbyte betraktas som prestandaproblem; större arbete delas upp, flyttas eller batchas.

Pan och zoom ska vara visuellt separerade från tung databehandling. Under aktiv kamerarörelse får systemet tillfälligt förenkla labels, skjuta upp collision-omräkning, dölja lågprioriterade edges och minska glow-kvalitet; när rörelsen avslutas återställs detaljnivån kontrollerat.

Öppning av inspector får inte starta force simulation, bygga om hela grafen, göra full datanormalisering eller orsaka märkbar freeze. Tung detaljdata i inspector laddas asynkront; selection och panelöppning ska kännas omedelbara.

7.16 Mobilbudget samt termisk och batterimässig hänsyn

Pixel 10 Pro och liknande mobiler använder **normal graph mode** med ungefär upp till 75–100 individuella synliga noder, begränsat antal labels, omkring 4–8 aktiva edge-animationer, reducerad glow och förenklade territories. Vid större scope används **summary mode** med project hubs, kritiska noder, aktiva workflows, selected path, counts och listvy. Över ungefär 150–200 relevanta mobila objekt bör grafen automatiskt gå mot tydligare aggregation.

Mobilgrafen ska inte hålla kontinuerlig force simulation, stark blur, många partiklar eller konstant 60 FPS-animation när inget händer. När systemet är lugnt går renderingen till vila — ambient livskänsla får inte innebära konstant hög GPU-belastning.

7.17 Prestandatelemetri

Under utveckling och kontrollerad produktion bör systemet mäta: FPS, frame time, long tasks, node count, edge count, synligt label count, collision time, simulation duration, snapshot duration, event ingestion rate, visual update rate, render backlog, memory trend, vald kvalitetsnivå, device class och degraded render count. Telemetrin får inte innehålla känsliga graph payloads.

7.18 Testscenarier och verktyg

Minst följande datasets ska finnas (de speglas i testplanen i kapitel 9): Scenario A — Small (cirka 50 noder, 100 edges, några aktiva workflows); Scenario B — Medium (cirka 300 noder, 1 000 edges, flera communities, simultana active paths); Scenario C — Large (cirka 1 000 noder, flera tusen potentiella edges, aggregation aktiverad, failures och approvals); Scenario D — Extreme source dataset (10 000+ lagrade objekt, endast summaries renderas, drilldown laddar relevant delmängd); Scenario E — Event spike (normal eventström, kort burst, failures och approvals mitt i bursten); Scenario F — Long session (minst 30 minuter med projektbyte, drilldown, freeze/resume, fullscreen, mobil och desktop).

Implementationen verifieras med riktiga profileringsverktyg: browserns Performance-panel, React-profilering där det är relevant, PerformanceObserver, memory snapshots, FPS/frame time-instrumentering, Lighthouse endast där dess mätningar är relevanta, Android browser-/enhetsprofilering samt manuella tester på MacBook Air M4 och Pixel 10 Pro. Syntetiska siffror utan riktig profilering är inte tillräckliga.

7.19 Acceptanskriterier

Performance budget är korrekt implementerad när:

- small graph känns nära 60 FPS på MacBook Air M4
- normal graf inte ligger ihållande under 30 FPS
- force simulation stoppas efter stabilisering
- idle-läget ger tydligt lägre belastning
- hover, selection och pan svarar direkt
- inspector öppnas utan full grafomräkning
- events batchas utan att kritiska events fördröjs orimligt
- samma event inte orsakar flera visuella animationer
- stora run-mängder aggregeras
- över 1 500 relevanta objekt används summary/drilldown
- mobilen visar en fokuserad graf snarare än desktopgrafen
- quality reduction tar bort effekter före statusinformation
- bakgrundsfliken inte fortsätter animera
- minnet stabiliseras under längre sessioner
- stora datasets testas utan att UI:t kraschar
- reduced motion och Static quality behåller all operativ information
- prestanda verifieras på riktig MacBook Air M4 och Pixel 10 Pro
- inga kritiska failures eller approvals försvinner på grund av optimering

7.20 Icke-mål

Performance budget kräver inte att alla enheter visar exakt samma visuella kvalitet, att 10 000 noder renderas samtidigt, att alla events får en egen animation, att mobilen uppnår desktopdensitet, att browsern håller 60 FPS under extrema datasets, att grafen alltid använder Web Worker eller att alla preliminära gränsvärden behålls efter verklig profilering.

Budgeten ska styras av mätning, inte stolthet.

Kapitelsammanfattning

Performance budget gör prestanda till ett kontrakt: mätbara mål för frame time, latens, laddning och stabilisering; fyra grafstorleksnivåer och fyra kvalitetsnivåer med automatisk degradering som alltid offrar presentation före information; samt budgetar för labels, motion, glow, events, minne och nätverk — allt verifierat på riktiga målenheter. Kapitel 8 beskriver i vilken ordning allt detta byggs.

Kapitel 8 — Implementationsfaser

Detta kapitel bygger på Implementation Phases v1.0. Det delar implementationen i sex kontrollerade faser — från audit av kontrakt till hardening och release — med tydligt scope, uttryckliga icke-mål, exit gates och stoppregler för varje fas, samt reglerna för Execution Replay och Graphify-artifactens leverans.

8.1 Beslutssammanfattning

Implementationen ska inte göras som en enda stor diff. Arbetet delas i sex kontrollerade faser:

1. **Audit & contracts** — verifiera nuvarande grafmotor, runtime-data, Graphify-artifact, statusar och säkerhetsmodell.
2. **Visual foundation** — bygg design tokens, nodformer, edge-språk, Atlas och stabil grundlayout, utan full realtime.
3. **Zoom, labels & interaction** — implementera semantic zoom, collision avoidance, drilldown, fullscreen och inspector.
4. **Live Operations truth layer** — koppla verkliga snapshots, events, stale state, approvals, failures och freeze.
5. **Accessibility, mobile & scale** — listvy, tangentbord, reduced motion, Pixel-layout, aggregation och kvalitetsnivåer.
6. **Hardening & release** — prestandamätning, säkerhetsgranskning, Codex-review, regressionstest och PR.

Varje fas måste verifieras innan nästa påbörjas. Ingen fas får använda falsk aktivitet för att demonstrera framtida funktionalitet.

8.2 Syfte

Fasindelningen ska undvika: en enorm svårgranskad diff; att visuell motion byggs före datakontrakt; att realtime blandas ihop med UI-animation; att projektisolering försvagas; att labels och prestanda lämnas till slutet; att mobil och accessibility byggs som efterhandslösningar; att Graphify-importen eller den befintliga MVP:n förstörs; att flera utvecklingsspår krockar i Git.

Varje fas ska ha tydligt scope, uttryckliga icke-mål, kunna testas separat, kunna pausas utan att main lämnas trasig och lämna grafen i ett användbart tillstånd.

8.3 Implementationsprinciper

Befintlig MVP bevaras. Den nuvarande implementationen är grundplattformen. Följande ska bevaras och vidareutvecklas: /intelligence/graph, System Map, Live Operations, disabled Execution Replay, autentiserade API-routes, project isolation, Graphify-import, sanitiserat graph contract, search, filter, inspector, zoom och pan, deterministisk layout, keyboard accessibility och runtime-data. Arbetet ska inte börja med en total omskrivning; en omskrivning får bara föreslås om auditen visar att nuvarande renderingsstack objektivt blockerar kraven.

Truth before motion. Operational motion implementeras först när källan är verifierad. Tillfälliga testfixtures får användas i utvecklings- och testmiljö, men de ska vara tydligt märkta, aldrig visas i produktion, inte blandas med verklig runtime-data och inte ligga bakom ett dolt produktionsflaggläge.

Små vertikala leveranser. Varje fas bör delas i mindre leveranser med plan, implementation, tester, visuell verifiering, prestandakontroll och diff-review.

Feature flags. Större delar bör kunna skyddas av interna feature flags när det minskar release-risk — till exempel new node renderer, semantic labels, live event layer, quality auto-degradation, mobile graph och Graphify artifact view. Flags får inte permanent ersätta riktig implementation eller städning.

Ingen backendmigration utan verifierat behov. Auditen ska först avgöra om nya tabeller eller migrations krävs. Nya schemaobjekt får inte skapas bara för att dokumenten beskriver en ideal framtidsmodell.

8.4 Fas 0 — Repository- och runtime-audit

Syftet är att skapa en verifierad bild av vad som redan finns innan någon betydande kod ändras.

Ska verifieras:

- **Git och repository:** uppdaterad main, ren working tree, relevant branchhistorik, aktuell grafimplementation, tidigare PR #44, pågående arbete som kan kollidera, nuvarande test- och buildkommandon.
- **Renderingsstack:** SVG, Canvas, WebGL eller hybrid; grafbibliotek; layoutmotor; zoom/pan-implementation; nod-, edge- och labelrenderer; hit testing; keyboardmodell; mobilstöd; worker-stöd.
- **Graph contract:** node types, edge types, statusfält, project scope, metadata, sanitization, Graphify-import, deterministiska positioner, backward compatibility.
- **Runtime:** tabeller, API-routes, Supabase Realtime, polling, runs, workflows, agents, approvals, incidents, outputs, Atlas-events, Manager-events, statusvärden och source of truth per entity.
- **Säkerhet:** authentication, authorization, project filtering, RLS, global Atlas access, cross-project data, approval actions, incident actions.
- **Graphify-artifact:** hur artifacten ska publiceras, var den byggs, hur Vercel får tillgång till den, fallback om artifacten saknas, om artifacten är statisk, genererad eller versionsstyrd.

Leverabler: repository audit, affected-file map, runtime source of truth-register, graph contract gap analysis, risk register, bekräftad testplan per fas samt en rekommendation om den befintliga grafmotorn kan behållas.

Exit gate Fas 0. Fasen är klar först när vi vet vad som finns, vad som saknas, vad som behöver ändras, vilka statusar som verkligen stöds, vilka migrations som eventuellt krävs och om branchens scope kan hållas säkert.

8.5 Fas 1 — Visual foundation

Syftet är att skapa den visuella grundarkitekturen utan att bygga full realtime-logik.

Omfattning: ett design token-system med centrala semantiska tokens för node size, shape, fill, stroke, glow, opacity, status, edges, labels, focus, quality samt dark/light mode; en nodrenderer med stöd för Atlas, project hub, Manager, agent, workflow, run, tool, approval, incident, output och cluster/summary; en edge renderer med stöd för de verifierade relationstyperna contains, reports to, delegates to, executes, assigned to, uses, produces, depends on, blocks och requires approval; Atlas visuella identitet med flerskiktsskärna, signaturpalett, dubbel halo, statisk reduced motion-variant och idle-animation som inte antyder arbete; projektterritorier med spatial separation, subtil atmosfär, projektaccent, stabil project hub och tydlig global Atlas-position; samt kontrollerad stabilisering med Atlas och project hubs som starka ankare, 2–3 sekunders stabilisering, stopp efter tidsgräns, ingen evig simulation och reset till deterministisk layout.

Fasen använder befintliga verkliga graph-data. Statusanimationer kan begränsas till statiska visuella states om runtime-kontraktet ännu inte är färdigt.

Icke-mål: full realtime subscription, freeze buffer, komplett event normalization, Execution Replay, avancerad mobilupplevelse och full prestandaautomation.

Exit gate Fas 1. Alla centrala nodtyper går att skilja visuellt; Atlas känns unik; project hubs är separerade; grafen stabiliseras och går till vila; light och dark mode fungerar; befintlig search och inspector är inte trasiga; projektisoleringen är oförändrad eller starkare.

8.6 Fas 2 — Zoom, labels och interaction

Syftet är att lösa det största nuvarande användbarhetsproblemet: informations- och labelröran.

Omfattning: semantic zoom med nivåerna portfolio overview, project/community overview, operational overview, detailed inspection och execution detail; en label engine med label priorities, selected + nearby, collision detection, density budget, truncation, automatisk offset, undantag för kritiska labels samt viewport- och inspector-avoidance; selection där enkelclick väljer, inspector öppnas, närrelationer framhävs, resten dimmas och ingen full layoutomräkning sker; drilldown initialt för project, workflow, agent och run — approval, incident och output läggs till när deras runtime-modeller är verifierade; focus och isolate med focus community, isolate project, isolate path, exit isolate, breadcrumbs och back history; kamerafunktionerna fit graph, fit path, focus node, reset view, reset all, stabil pan och zoom samt riktig fullscreen; och URL-state som representerar säkra delar av current view, projekt, selection, filter, drilldown och isolate.

Exit gate Fas 2. Labels överlappar betydligt mindre; låg zoom visar endast landmarks och kritiska signaler; selection fungerar utan layoutkast; drilldown har breadcrumb och back; fullscreen har komplett exit och controls; browser refresh kan bevara viktig säker kontext; tangentbordets befintliga stöd har inte försämrats.

8.7 Fas 3 — Live Operations truth layer

Syftet är att göra Live Operations verkligt live utan spekulativa statusar.

Omfattning: tidsstämplad initial snapshot med projekt- och authority-scope, freshness och source metadata; event normalization med ett normaliserat internt eventkontrakt baserat på verkliga källor (event-ID, type, project, entity, occurred at, sequence, correlation, causation, status transition, source — endast fält som stöds eller behövs implementeras); hybrid realtime med

Supabase Realtime där det är verifierat, polling där realtime saknas, periodisk reconciliation och heartbeat/connection health; koppling av verkliga operational states för runs, workflows, agents, approvals, incidents, outputs samt Atlas och Manager där stöd finns; motion aktiverad endast för verkliga händelser — running execution, delegation, tool use, output, approval block, failure, retry — och ingen motion för unsupported states; connection states (connecting, live, degraded, reconnecting, offline, snapshot only); stale och unknown med stoppad aktiv motion, last confirmed, tydlig statisk status och reconciliation; freeze med fryst synligt state, eventbuffer, count, return to live, jump to latest samt deduplicering och reconciliation; och en activity rail med verifierade events, projektscope, fokus av nod/path och inga spekulativa texter.

Exit gate Fas 3. "Live" visas endast med verifierad health; stale running slutar animera; reconnect hämtar eller verifierar snapshot; eventdubletter skapar inte dubbla animationer; gamla events skriver inte över ny status; andra projekt läcker inte in; failure och approval har egna sanningsmodeller; freeze tappar inte kritiska events.

8.8 Fas 4 — Accessibility, responsive och list view

Syftet är att göra grafen praktiskt användbar på fler enheter och utan beroende av canvasens visuella presentation.

Omfattning: en fullvärdig listvy synkroniserad med search, filter, selection, project scope, live/frozen, inspector, status och actions; verifierat och utökat tangentbordsstöd för nodnavigation, relationsnavigation, Enter, Space, Escape, zoom, search, focus, isolate och fullscreen exit; screen reader med semantisk representation, status, relationer, counts, kritiska announcements och begränsad live region; reduced motion med statisk layout, inga partiklar, inga upprepade pulser, statiska statusmönster och korta eller omedelbara kameraövergångar; high contrast med starkare outlines, lägre transparens, minskat glow, tydligare labels och edge patterns; samt mobil med attention-first home state, failures, approvals, active runs, mindre graf, bottom sheet-inspector, list/graph-toggle, touch targets, pinch/pan och explicit drilldown.

Exit gate Fas 4. Huvudflöden fungerar utan mus; reduced motion behåller all information; failure och approval kan skiljas utan färg; listvyn fungerar utan canvas; Pixel 10 Pro får användbar interaktion; ingen kritisk action kräver canvasgest; textzoom och focus fungerar.

8.9 Fas 5 — Scale och quality adaptation

Syftet är att säkerställa att grafen fungerar från små system till stora datamängder.

Omfattning: aggregation med run clusters, output summaries, community summaries, incident summaries och approval summaries med undantag för kritiska approvals; kvalitetsnivåerna High, Medium, Low, Static och Auto; automatisk degradering baserad på node count, edge count, FPS, long tasks, device class, reduced motion, viewport och event rate; en render scheduler med central hantering av requestAnimationFrame, background tab, pause motion, idle state, operational motion och quality mode; Web Worker som införs endast om profilering visar behov för force simulation, spatial index, aggregation eller collision preprocessing; samt tester av extrem data med 10 000+ lagrade objekt, summaries initialt, progressiv drilldown och ingen full detaljrendering.

Exit gate Fas 5. Små grafer får full upplevelse; stora grafer aggregeras automatiskt; kritisk information överlever degradering; bakgrundsfliken slutar animera; long session visar ingen uppenbar minnesläcka; mobilen går till summary mode innan den blir oanvändbar.

8.10 Fas 6 — Hardening och release

Syftet är att verifiera att den betydande förändringen är säker, mätbar och redo att mergas.

Omfattning: **regression** — verifiera System Map, Live Operations, disabled Execution Replay, Graphify-import, API-auth, project isolation, search, filter, inspector, keyboard, zoom/pan och deterministisk layout; **säkerhet** — authentication, project authorization, RLS, API scope, realtime subscriptions, approval authority, incident actions, payload minimization, URL-state och cross-project relations; **prestanda** — tester på MacBook Air M4, stationär dator, Pixel 10 Pro och vanlig kontorslaptop där det är möjligt, med datasets small, medium, large, extreme source, event spike och long session; **accessibility** — tangentbord, screen reader, reduced motion, high contrast, 200 procents textzoom, mobil och list-only fallback; **visuell QA** — dark mode, light mode, Atlas, projektterritorier, labels, approvals, failures, stale/offline, fullscreen samt screenshots och video som bevis; **Codex-review** — Codex granskar den betydande diffen före merge med fokus på regressionsrisk, säkerhet, projektisolering, event correctness, race conditions, prestanda, accessibility, dead code och testluckor.

Exit gate Fas 6. Ingen merge förrän tester passerar, builden passerar, audit findings är hanterade, Codex-review är genomgången, kvarvarande risker är dokumenterade, användaren har godkänt resultatet och PR:n är fokuserad och begriplig.

8.11 Execution Replay

Full Execution Replay ingår inte i denna implementation. Den disabled fliken ska ligga kvar.

Implementationen ska däremot skapa fundamentet: event persistence, ordering, correlation, causation, attempts, approval history, incident history, output links och execution detail view. Replay blir senare ett separat epic eller en separat fas efter att Live Operations är stabilt.

8.12 Graphify-artifact

Publiceringen av System Maps Graphify-artifact ska behandlas tidigt i audit och leveransplan. Det ska avgöras om artifacten byggs lokalt, genereras i CI, lagras versionsstyrt, hämtas från skyddad storage samt byggs per commit eller release.

Artifacten får inte: innehålla secrets; kringgå projektisolering; vara en odokumenterad manuell fil; göra Vercel-deployment beroende av en utvecklarens lokala dator; publiceras utan versionsidentifiering; få grafen att krascha om artifacten saknas eller är ogiltig.

System Map ska ha ett tydligt fallback-state när artifacten inte finns: en förklaring om varför kartan saknas, senaste kända artifact-version där det är säkert, möjlighet att försöka läsa in igen samt fortsatt åtkomst till Live Operations och listvyn.

8.13 Fasgränser och stoppregler

Varje fas ska avslutas med en sammanfattning av genomfört arbete, ändrade filer, tester och resultat, visuella bevis där det är relevant, kända begränsningar, säkerhets- och prestandabedömning samt Git-status.

Nästa fas får inte starta om: föregående fas lämnar kända kritiska regressioner; projektisoleringen inte är verifierad; build eller obligatoriska tester misslyckas; implementationen avviker från runtime-sanningen; diffen har vuxit utanför överenskommet scope.

8.14 Övergripande acceptanskriterier

Implementationsfaserna är korrekt genomförda när:

- arbetet har följt den definierade fasordningen
- varje fas har verifierats innan nästa startat
- befintlig MVP har bevarats eller förbättrats
- ingen falsk runtime-aktivitet visas
- System Map och Live Operations behåller tydliga separata roller
- projektisolering är backend-verifierad
- Graphify-artifacten har en reproducerbar publiceringsmodell och fallback
- labels, interaktion, realtime, accessibility och skala har införts kontrollerat
- Execution Replay fortfarande är utanför scope
- Codex har granskat den betydande slutdiffen
- ingen merge har skett före full verifiering och grundarens godkännande

Kapitelsammanfattning

Six faser — audit, visual foundation, zoom och labels, truth layer, accessibility och skala, hardening — bygger grafen i verifierbara steg där varje fas har exit gates och stoppregler. Den befintliga MVP:n bevaras, sanning byggs före rörelse, Execution Replay förblir utanför scope och Graphify-artifacten får en reproducerbar leverans. Kapitel 9 definierar hur varje fas testas.

Kapitel 9 — Testplan

Detta kapitel bygger på Test Plan v1.0. Det definierar testprinciper, testnivåer, testmiljöer, sju testdatasets, fasvisa tester, approval-, failure- och isolationstester, accessibility-, mobil- och prestandatester, severity-nivåer, release gate och testrapportens innehåll.

9.1 Beslutssammanfattning

Testplanen låser följande: varje implementationsfas testas separat, inte bara slutresultatet; verkliga runtime-kontrakt testas före visuella animationer; System Map och Live Operations får egna testsviter; projektisolering och RLS är blockerande releasekrav; grafen testas med small, medium, large och extreme datasets; realtime testas för dubletter, fel ordning, reconnect, stale data och eventspikar; failures, approvals och incidents måste behålla korrekt historik; labels testas visuellt för överlapp och prioritering; accessibility testas med tangentbord, screen reader, reduced motion och textzoom; desktop testas främst på MacBook Air M4 och mobil på Pixel 10 Pro; prestanda verifieras med riktiga profileringsverktyg; screenshots, video och mätvärden sparas som testbevis; testdata blandas aldrig med produktionsdata; en release godkänns inte med kritiska säkerhets-, sannings- eller regressionsfel; full Execution Replay ligger fortfarande utanför scope.

9.2 Syfte och principer

Testplanen ska säkerställa att Intelligence Graph visar korrekt information, reagerar förutsägbart, inte läcker data mellan projekt, förblir användbar vid stora datamängder, fungerar på desktop och mobil, är tillgänglig utan mus, inte skapar falsk live-känsla, degraderar säkert vid låg prestanda och bevarar befintlig MVP-funktionalitet.

En graf som är tekniskt korrekt men oläslig är inte godkänd. En graf som är visuellt imponerande men visar fel status är heller inte godkänd.

Principer:

- **Truth före presentation.** Testerna verifierar först datakälla, projektscope, eventidentitet, statusövergång, tidsstämpel och source of truth — därefter animationen.
- **Testa fasvis.** Varje implementationsfas har egna exit-tester; nästa fas får inte användas för att dölja problem i föregående.
- **Reproducerbara tester.** Testresultat ska kunna återskapas genom dokumenterade kommandon, versionsstyrda fixtures, seedade datasets, tydliga miljökrav och sparade testbevis.
- **Testmiljö separerad från produktion.** Fixtures och syntetiska events används endast i lokal utvecklingsmiljö, testmiljö, preview deployment och automatiserade tester — aldrig blandade med verkliga produktionshändelser.

9.3 Testnivåer

- **Unit tests** för isolerad logik: statusmappning, eventdeduplicering, ordering, freshness, quality selection, labelprioritering, aggregation, URL-state, project scope och graph normalization.
- **Component tests** för nodrendering, edgerendering, inspector, toolbar, activity rail, listvy, filter, search, statusindikatorer och connection state.
- **Integration tests** för flöden mellan API och graf, snapshot och realtime, selection och inspector, graf och listvy, filter och project scope, approval action och backendbekräftelse samt reconnect och reconciliation.
- **End-to-end tests** för användarflöden i riktig browser.
- **Visuell QA** för former, färger, glow, labels, territories, dark/light mode, responsiv layout, fullscreen och reduced motion.
- **Manuell device-verifiering** på verkliga enheter för det automatiska tester inte fångar: touchkänsla, termisk belastning, batteripåverkan, trackpad-zoom, fullscreen, browserbeteende och faktisk läsbarhet.

9.4 Testmiljöer

Minst följande miljöer används: **lokal utveckling** för snabb iteration och fixtures; **preview deployment** för realistisk browser- och deploymentverifiering; **testprojekt** — ett isolerat projekt med testagents, testworkflows, test-runs, testapprovals och testincidents; **produktion** — endast kontrollerad smoke test efter godkänd deployment. Produktion får inte användas för destruktiva eller syntetiska belastningstester.

9.5 Testdatasets

- **Dataset A — Small:** cirka 50 noder, 100 edges, 2–4 projekt, några agents och workflows, en running path, ett failure, en approval. Används för visuell kvalitet, full motion, normal användning och accessibility.
- **Dataset B — Medium:** cirka 300 noder, 1 000 edges, flera communities, samtidiga runs, flera approvals och incidents. Används för semantic zoom, label density, clustering, stabilisering och quality Medium.
- **Dataset C — Large:** cirka 1 000 synliga eller relevanta noder, flera tusen relationer, många historiska runs. Används för aggregation, edgeprioritering, quality Low, prestanda och listvy.
- **Dataset D — Extreme source:** 10 000+ lagrade objekt, endast summaries initialt, progressiv drilldown, begränsad klientladdning. Verifierar att grafen inte försöker rendera allt.
- **Dataset E — Event spike:** stabil normal eventström, kort burst, dubletter, events i fel ordning samt failure och approval under bursten.
- **Dataset F — Failure and approval:** open incident, acknowledged incident, resolved incident, pending approval, approved, rejected, expired och retry attempts.
- **Dataset G — Security scope:** minst två projekt med objekt som har liknande namn och ID:n — används för att upptäcka cross-project-läckor.

9.6 Fasvisa tester

Fas 0 — Audit och contracts. Verifiera aktuell grafmotor, graph contract, node types, edge types, runtime-statusar, API-routes, Supabase-tabeller, Realtime subscriptions, polling, RLS och Graphify-artifactflödet. Godkänt när varje entity har dokumenterad source of truth, verkliga statusvärden är identifierade, projektisoleringen är verifierad, eventkällorna är kända och gap mellan dokumentation och implementation är dokumenterade.

Fas 1 — Visual foundation. Verifiera att användaren visuellt kan skilja Atlas, project hub, Manager, agent, workflow, run, tool, approval, incident, output och cluster. För Atlas: största och tydligaste noden, unik identitet, rätt halo, idle-motion som inte antyder aktiv execution och fungerande reduced motion-variant. För projektterritorier: tydlig separation, subtil accent, inga hårda ogenomträngliga bubblor och Atlas som globalt ankare. För stabilisering: grafen stabiliseras inom budget, simulationen stoppas, positionerna förblir stabila, reload ger reproducerbar grundlayout och reset återställer baslayouten.

Fas 2 — Zoom, labels och interaction. För varje zoomnivå verifieras vilka nodtyper, labels och edges som visas, vilka objekt som aggregeras och att positionerna inte hoppar. Labels: ingen normal labelöverlappning; Atlas och projektnamn syns; selected, failure och approval prioriteras; långa namn trunkeras med fullständigt namn i inspector; labels undviker controls och inspector. Selection: enkelclick väljer, inspector öppnas, status bibehålls, närliggande edges framhävs, layouten räknas inte om och Escape avmarkerar. Drilldown testas för project, workflow, agent och run med breadcrumb, back, kamerastate, selection, filter och isolate state. Fullscreen: entry, exit, Escape, inspector toggle, search, filter, fit/reset, pause motion och freeze live view.

Fas 3 — Realtime truth. Initial snapshot: timestamp, projektscope, freshness, source, aktiva runs, approvals och incidents. Realtime event: uppdaterar rätt entity och rätt projekt, påverkar inte andra projekt, visar rätt status och trigger högst en animation. Dubbletter: samma event skickas flera gånger — state appliceras en gång, animationen spelas en gång och activity rail dubblas inte. Out-of-order: completed följt av ett sent running-event — entityn förblir completed, det sena eventet kan finnas i historiken och den aktuella statusen backar inte. Stale data: simulera förlorad anslutning — live-indikatorn försvinner, status blir reconnecting eller stale, operational motion stoppas och senast bekräftad tid visas. Reconnect: ny snapshot, deduplicering, reconciliation, inga hundratals catch-up-animationer och kritiska missade events syns. Freeze: grafens state fryses, events buffras, event count visas, zoom och inspector fortsätter fungera och return to live återställer rätt state.

9.7 Approval-tester

Hela livscykeln testas: requested, pending, approved, rejected, expired samt cancelled där det stöds. Verifiera att approvalen är ett eget objekt, att den blockerade executionen visas, att projektet är korrekt, att en action kräver explicit knapp, att UI visar submitting, att state ändras först efter serverbekräftelse, att fel återställer tidigare state och att historiken bevaras.

■ Ett nodklick får aldrig godkänna en approval.

9.8 Failure- och incidenttester

Verifiera skillnaden mellan failure event och incidentobjekt. Testa: run failed, incident open, acknowledged, resolved, retry, retry failed och retry completed. Förväntat: den ursprungliga failed-runnen förblir failed; resolution skriver inte om historiken; nya attempts visas separat; incidentstatus ändras korrekt; failure fortsätter vara spårbart.

9.9 Projektisolering

Detta är ett blockerande releasekrav.

Verifiera: projekt A ser inte projekt B:s privata noder; projekt A får inte projekt B:s events; search respekterar projektgränser; activity rail respekterar projektgränser; inspector kan inte läsa ett annat projekts payload; URL-manipulation ger inte åtkomst; subscriptions filtreras backend-side; RLS nekar otillåtna reads; actions nekar otillåten authority. Atlas Global View testas separat med korrekt behörighet.

9.10 System Map- och Live Operations-tester

System Map: Graphify-artifacten laddas; versionen visas eller kan spåras; ogiltig artifact hanteras; saknad artifact ger tydligt fallback-state; inga secrets finns i artifacten; artifacten kan reproduceras; layouten är stabil; runtime-status dominerar inte arkitekturvyn. System Map ska förbli användbar även när realtime är nere.

Live Operations: aktiva runs, running agents, approvals, failures, incidents, stale state, connection state, activity rail, freeze och selected execution path. Live Operations utan aktivitet ska vara lugn — ingen fejkad edge-motion får visas.

9.11 Accessibility-tester

Tangentbord: hela huvudflödet med Tab, piltangenter, Enter, Space, Escape, zoomkommandon, search, focus och isolate. Screen reader: aktuell vy, antal failures, antal approvals, selected node, status, relationer, actions och drilldown-kontext. Reduced motion: inga partiklar, inga upprepade pulser, inga långa kameraflygningar, samma statusinformation och samma actions. Färg och kontrast: dark mode, light mode, high contrast, färgblindhetssimulering och status utan färg. Textzoom: minst 100, 150 och 200 procent.

9.12 Mobiltester

Primär enhet: Google Pixel 10 Pro. Verifiera initial mobile view, list/graph-toggle, tap selection, pinch zoom, pan, inspector bottom sheet, fullskärmsdetalj, back gesture, touch targets, filter, search, approvals, failures och summary mode. Testa även portrait, landscape, synlig browser chrome, öppet tangentbord, reduced motion och battery saver där det är möjligt.

9.13 Prestandatester

Mät FPS, frame time, long tasks, simulation duration, label collision time, node count, edge count, memory trend, event lag, render backlog och quality level.

Mål per dataset: Small — nära 60 FPS på MacBook Air M4; Medium — stabil interaktion och automatisk kvalitetsanpassning; Large — aggregation och fortsatt användbarhet; Extreme — summaries utan full detaljrendering; Event spike — ingen UI-freeze, kritiska events prioriteras och progress-events batchas; Long session — minst 30 minuter med zoom, projektbyte, drilldown, freeze/resume, fullscreen och live-events, där minnet inte får växa utan gräns.

9.14 Regressionstest

Följande befintliga funktioner måste fortsätta fungera: /intelligence/graph, System Map, Live Operations, disabled Replay, authentication, Graphify-import, project isolation, search, filter, inspector, zoom, pan, deterministisk layout, keyboard navigation, runtime-data och API-routes.

9.15 Error och fallback

Testa snapshot error, realtime error, båda nere, ogiltigt event, okänd schema version, tom graf, saknad artifact, renderingsfel, network offline, timeout och backend action failure. Förväntat: tydligt textmeddelande, ingen krasch, ingen falsk status, listvy när grafen inte fungerar samt säkra actions och retry där det är relevant.

9.16 Visuella testbevis

För varje större leverans sparas screenshots före och efter, dark mode, light mode, låg zoom, normal zoom, selected state, failure, approval, offline/stale, desktop, mobil och reduced motion. Video används för stabilisering, semantic zoom, execution flow, reconnect, freeze/resume, mobilgester och quality degradation.

9.17 Automatisering kontra manuella tester

Automatisera främst contracts, status transitions, project scope, event ordering, deduplicering, filter, URL-state, API authorization och regressionsflöden. Verifiera manuellt visuell hierarki, labelkvalitet, touchkänsla, glow, motion, faktisk läsbarhet, termisk mobilpåverkan och känslan av command center.

Automatiska tester kan inte ensamma avgöra om grafen känns bra.

9.18 Severity-nivåer

- **Blocker:** data från fel projekt; approval kan utföras oavsiktligt; felaktig runtime-status; produktionen kraschar; auth eller RLS kringgås; dataförlust; falsk live-status.
- **Critical:** failures eller approvals döljs; reconnect ger fel state; incidenthistorik skrivs om; stora regressionsfel; grafen oanvändbar på primär enhet.
- **Major:** omfattande labelöverlapp; drilldown eller fullscreen trasigt; stora prestandaproblem; accessibility-huvudflöde trasigt.
- **Minor:** mindre visuella avvikelser; lågprioriterad animation; kosmetisk spacing; ovanliga edge cases utan säkerhetspåverkan.

9.19 Release gate

Ingen merge eller release om något av följande finns: öppet blocker-fel; öppet critical-fel utan uttryckligt beslut; misslyckad build; ej verifierad projektisolering; falsk runtime-status; approvals som inte är serverbekräftade; okontrollerad minnesökning; graf som inte fungerar på primär desktop; trasiga kritiska mobilflöden; ohanterad Codex-review; saknade testbevis för större ändringar.

Major-fel rättas normalt före release eller dokumenteras och godkänns uttryckligen.

9.20 Testrapport

Slutrapporten ska innehålla: branch och commit, miljö, testdatum, testade enheter, testkommandon, automatiska resultat, manuella resultat, performance metrics, screenshots och video, säkerhetsresultat, kvarvarande problem, riskbedömning, release-rekommendation och slutlig Git-status.

9.21 Acceptanskriterier

Testplanen är genomförd när:

- varje fas har verifierats separat
- runtime truth är testad före motion
- projektisolering är verifierad i backend
- System Map och Live Operations har separata testresultat
- labels är testade vid flera zoomnivåer
- dubletter och out-of-order events hanteras
- stale, reconnect och freeze fungerar
- approvals kräver explicit och bekräftad action
- failure- och incidenthistorik bevaras
- small, medium, large och extreme datasets har testats
- desktop och Pixel 10 Pro har verifierats
- tangentbord, reduced motion och listvy fungerar
- prestandamätningar finns
- regressionstester passerar
- visuella bevis finns
- inga blockerande problem återstår
- Codex-review är genomförd före merge

Kapitelsammanfattning

Testplanen testar sanning före presentation, fas för fas, med sju datasets från small till security scope. Projektisolering och RLS är blockerande releasekrav, approvals kräver serverbekräftad explicit action, historiken för failures och incidents är orörbar och release sker endast genom en definierad gate. Kapitel 10 beskriver hur allt detta levereras säkert genom Git.

Kapitel 10 — Git- och leveransplan

Detta kapitel bygger på Git & Delivery Plan v1.0. Det styr startläge, branch och worktree, commitstrategi, push-policy, migrationspolicy, Graphify-artifactens leveransflöde, preview deployment, PR-strategi, Codex-review, merge gate, rollback, dokumentation efter implementation och stoppregler.

10.1 Beslutssammanfattning

Planen låser följande: arbetet börjar alltid från uppdaterad och ren main; en egen branch används — feat/intelligence-graph-visual-motion; helst används en separat Git worktree; ingen kod skrivs innan Fas 0-auditen är klar; arbetet delas efter de godkända implementationsfaserna; varje fas får egna fokuserade commits; ingen direkt commit eller push till main; ingen merge innan build, tester, security review och Codex-review är klara; databasmigrationer skapas endast efter verifierat behov; Graphify-artifaken får ett reproducerbart CI- eller repoflöde; preview deployment används för visuell och funktionell verifiering; PR:n innehåller testbevis, risker och rollbackplan; Execution Replay ingår inte; om diffen blir för stor delas arbetet i flera PR:er; grundaren godkänner slutresultatet före merge.

10.2 Syfte

Planen ska säkerställa att Intelligence Graph kan implementeras utan att störa andra pågående Omnira-spår, skriva direkt till main, skapa svårgranskade jättecommits, blanda audit, migrations och UI utan kontroll, lämna repositoryt i ett oklart tillstånd, skapa dolda produktionsberoenden eller mergas innan säkerhet och runtime-sanning verifierats.

10.3 Startläge

Innan arbetet börjar verifieras att main är uppdaterad och ren: git checkout main, git pull origin main och git status ska visa "On branch main ... nothing to commit, working tree clean".

Om working tree inte är ren stoppas implementationen tills filerna har identifierats och hanterats. Ingen befintlig lokal ändring får raderas eller flyttas utan uttryckligt beslut.

10.4 Branch och worktree

Rekommenderad branch: **feat/intelligence-graph-visual-motion**.

Branchens scope omfattar: audit och kontraktsskartläggning, visual foundation, zoom- och labelmotor, interaction, Live Operations truth layer, accessibility och mobil, prestanda och quality adaptation, hardening samt Graphify-artifactens leverans.

Ingår inte: full Execution Replay, andra Atlas-epics, generell Memory-arkitektur, Executive Intelligence-implementation, orelaterad cleanup eller bred designrefaktorerings utanför grafen.

Eftersom flera stora Omnira-spår kan pågå parallellt rekommenderas en separat worktree, till exempel: git worktree add ../Omnira-intelligence-graph -b feat/intelligence-graph-visual-motion

origin/main. Fördelar: huvudrepositoriet hålls stabilt, mindre risk att byta branch med lokala ändringar, enklare parallellt arbete, tydligare Git-status och lägre risk att andra epics blandas in. Exakt sökväg anpassas efter den lokala repositorystrukturen.

10.5 Fas 0 före implementation

Det första arbetet på branchen ska vara repository audit, runtime audit, affected-file map, source of truth-register, graph contract gap analysis, Graphify-artifactanalys, risk register och bekräftad testmatris.

Ingen större produktionskod skrivs innan auditresultatet är granskat. Fas 0 får inte automatiskt övergå till implementation i samma körning.

10.6 Commitstrategi och commitgates

Commits ska vara små, fokuserade och läsbara. Föreslagen struktur:

- docs(graph): add runtime and repository audit
- refactor(graph): introduce semantic visual tokens
- feat(graph): add distinct node and edge renderers
- feat(graph): implement semantic zoom and label priorities
- feat(graph): add drilldown and isolate navigation
- feat(graph): add verified live snapshot model
- feat(graph): add realtime reconciliation and stale states
- feat(graph): add accessible synchronized list view
- feat(graph): add responsive mobile graph experience
- perf(graph): add aggregation and adaptive quality
- test(graph): add graph truth and interaction coverage
- chore(graph): harden Graphify artifact delivery

Commits får inte blanda migrations och stora visuella ändringar utan skäl, innehålla genererade filer som inte behövs, innehålla debugloggar eller secrets, innehålla orelaterad formattering eller av misstag ändra andra epics.

Innan en fas committas kontrolleras minst: git diff, git diff --stat, relevanta tester, typecheck, lint där det är tillämpligt, build eller relevant delbuild, inga secrets, inga orelaterade filer och korrekt projektisolering. Varje fasrapport anger vilka filer som ändrats och varför, vilka tester som körts, vad som ännu inte är implementerat och aktuell Git-status.

10.7 Push-policy

Push får endast ske till feature-branchen: git push -u origin feat/intelligence-graph-visual-motion. Push till main är inte tillåten. Force push undviks efter att andra har börjat granska branchen; om historiken måste korrigeras dokumenteras det tydligt.

10.8 Databasmigrationer

Migrationer får endast skapas när Fas 0 visar ett verifierat behov. Innan en migration skapas granskas befintligt schema, migrationshistorik, RLS-påverkan, rollback och kompatibilitet med pågående epics.

Migrationer ska vara additiva när det är möjligt, vara project-scoped, ha rätt RLS, testas lokalt eller i säker testmiljö och aldrig appliceras direkt i produktion utan uttryckligt godkännande.

Ingen migration får skapas endast för att matcha ett teoretiskt dokumentkontrakt.

10.9 Graphify-artifactens leverans

Graphify-artifaken ska få ett reproducerbart flöde, i föredragen ordning: verifiera hur artifaken genereras, definiera sanitization, definiera version, definiera lagringsplats, automatisera i CI eller dokumenterad releaseprocess, verifiera Vercel och implementera fallback.

Artifaken ska kunna kopplas till commit-SHA, build, schema version och generated timestamp. Den får inte vara beroende av att en viss lokal Mac är tillgänglig.

10.10 Preview deployment

Innan merge ska feature-branchen få en preview deployment som används för System Map, Live Operations, dark och light mode, labels, fullscreen, realtime, stale och reconnect, mobil, accessibility, prestanda och Graphify-artifaken. Preview använder säker testdata eller korrekt behörig verklig data; syntetiska events är tydligt separerade från produktion.

10.11 PR-strategi och PR-beskrivning

Två modeller finns. **Modell A — en sammanhängande PR** används endast om diffen förblir rimlig och lättgranskad. **Modell B — flera staplade PR:er** rekommenderas om implementationen blir stor, till exempel: audit och visual foundation; zoom, labels och interaction; realtime truth layer; accessibility och responsive; scale, hardening och artifact delivery. Varje PR måste lämna produkten stabil, och en del-PR får inte exponera en falsk "Live"-upplevelse.

PR:n ska innehålla: **Summary** (vad som byggts); **Scope** (vad som ingår och inte ingår); **Architecture** (renderer, graph contract, runtime sources, realtime-strategi, projektisolering, artifact delivery); **Testing** (testkommandon, automatiska och manuella resultat, device tests, performance metrics); **Visual evidence** (före/efter, screenshots, video, dark/light, desktop/mobil, reduced motion); **Security** (authentication, authorization, RLS, subscriptions, payloads, approval authority); **Risks** (kvarvarande risker och begränsningar); **Rollback** (hur ändringen kan inaktiveras eller återställas).

10.12 Codex-review

Innan merge granskar Codex den betydande diffen, med särskild kontroll av runtime correctness, event ordering, deduplicering, stale state, projektisolering, RLS, approval actions, race conditions, memory leaks, render loops, accessibility, mobilregressioner, Graphify-artifakens säkerhet, testluckor och orelaterade ändringar.

Alla findings klassificeras som blocker, critical, major, minor, accepted risk eller false positive. Blocker och critical löses före merge om inget mycket tydligt och dokumenterat undantag beslutas.

10.13 Merge gate och mergeform

Merge är tillåten först när: Fas 0–6 är genomförda enligt överenskommet scope; builden passerar; obligatoriska tester passerar; projektisolering är verifierad; RLS är verifierad; realtime visar verifierad sanning; Graphify-artifacten fungerar eller har säker fallback; Pixel 10 Pro-flödena är testade; MacBook Air M4 är prestandatestad; accessibility-huvudflödena fungerar; Codex-review är hanterad; PR-riskerna är dokumenterade; grundaren har godkänt resultatet.

Merge sker genom GitHub-PR, inte genom oannonserad lokal merge till main.

Föredragen mergeform: squash merge för små fokuserade PR:er; merge commit för större fasindelade PR:er där historiken har värde. Valet görs konsekvent med repositoryts etablerade praxis. Efter merge verifieras att main är ren: git checkout main, git pull origin main, git status.

10.14 Rollback

Rollbackstrategin definieras innan release. Möjliga lager: feature flag av; återgå till tidigare renderer; stäng realtime-lagret; använd snapshot only; använd listvyn; revert PR; återställ artifact-version; rollback av migration endast enligt säker migrationsplan.

Fallback ska prioritera korrekt information framför full visuell upplevelse.

10.15 Dokumentation efter implementation

Efter färdig implementation uppdateras: README för Intelligence Graph, graph contract, runtime source of truth-register, Graphify-artifactprocessen, feature flags, kvalitetsnivåer, testkommandon, runbook för stale/offline/reconnect, kända begränsningar och Execution Replay-förutsättningar.

De godkända styrdokumenterna skrivs inte om i efterhand för att dölja avvikelser — avvikelser dokumenteras öppet.

10.16 Stoppregler

Arbetet pausas om: main eller feature-branchen inte är ren vid start; andra epics har ändrat samma kärnfiler utan koordinering; projektisolering inte kan verifieras; verkliga runtime-källor motsäger planens antaganden; migrationens konsekvenser är oklara; diffen växer kraftigt utanför scope; prestandan kräver större renderer-omskrivning; Graphify-artifacten riskerar secrets; tester visar falsk runtime-status; blocker eller critical regression upptäcks. Vid stopp tas en rapport fram innan nytt arbete fortsätter.

10.17 Slutleverans

Slutleveransen innehåller: färdig feature branch, fokuserad PR eller PR-serie, komplett audit, testresultat, prestandamätningar, screenshots och video, säkerhetsbedömning, Codex-review, riskregister, rollbackplan, dokumentationsuppdateringar och ren Git-status.

10.18 Acceptanskriterier

Git- och leveransplanen är korrekt genomförd när:

- arbetet startade från ren och uppdaterad main
- egen feature branch eller worktree användes
- Fas 0 slutfördes före större implementation
- commits är fokuserade
- inga orelaterade ändringar följde med
- ingen direkt push skedde till main
- migrations skapades endast vid verifierat behov
- Graphify-artifaken har reproducerbar leverans
- preview deployment verifierades
- PR innehåller test- och visuella bevis
- Codex-review är hanterad
- rollback är definierad
- grundaren godkände resultatet före merge
- slutlig main är ren efter merge

Kapitelsammanfattning

Leveransplanen gör implementationen säker: ren main som startläge, en dedikerad feature branch i egen worktree, audit före kod, fokuserade commits med gates, migrations endast vid verifierat behov, reproducerbar Graphify-artifact, preview deployment, en beviskravande PR, Codex-review och en strikt merge gate med definierad rollback. Därmed är arkitekturens tio delar kompletta — bokens avslutande del konsoliderar invarianter, beroenden och styrning.

Ordlista

Term	Betydelse
Atlas	Omniras centrala intelligens och grafens största, mest unika nod. Observerar projekten, resonerar, delegerar och kommunicerar med Manager.
Manager	Executive koordineringsnod nära Atlas. Hanterar styrning och delegation; kan vara global eller projektspecifik med korrekt scope.
System Map	Grafens arkitekturläge: visar hur Omnira är uppbyggt — projekt, agents, workflows, tools, outputs och relationer. Stabil och nästan stilla.
Live Operations	Grafens runtime-läge: visar vad Omnira gör just nu — aktiva runs, approvals, failures och execution paths.
Execution Replay	Framtida läge som återberättar en historisk execution som en förenklad scen. Ingår inte i nuvarande implementationsomfattning.
Project hub	Ett projekts centrala nav och ankare för dess agents, workflows, tools, runs, approvals, outputs och incidents.
Community	Funktionell gruppering av noder — i System Map inom projekt (t.ex. Publishing), i Live Operations kring aktiv execution.
Workflow	Operativt nav inom ett projekt som organiserar arbete; representeras som rundad hexagon. Konfigurationsstatus skiljs från run-status.
Agent	Individuell exekverande entitet; cirkulär nod. Visas som executing endast med verifierat runtime-stöd.
Run	Den primära execution-instansen; liten temporär ringnod som bär det mest konkreta runtime-tillståndet. Aggregeras vid hög volym.
Tool	Funktionell systemresurs (API, databas, MCP m.m.); liten teknisk fyrkant/capsule. Visar aldrig credentials eller secrets.
Output	Producerad artefakt (artikel, rapport, fil m.m.); liten dokumentlik capsule.
Approval	Eget objekt med livscykeln requested → pending → approved/rejected/expired/cancelled. Diamond-form, amber/guld. Kräver explicit, serverbekräftad action.
Failure	Immutabelt operativt faktum: en execution misslyckades. Skrivs aldrig om i efterhand.
Incident	Hanteringsobjekt kopplat till failures, med livscykeln open → acknowledged → resolved. Ändrar aldrig den underliggande run-statusen.
Snapshot	Tidsstämplat, sammanhängande senast känt tillstånd. Grund för första render, reconnect och reconciliation. Inte automatiskt live.
Realtime	Push-baserade verifierade events som förändrar state. Används för run-status, approvals, failures m.m.
Polling	Periodisk hämtning för källor utan realtime samt för consistency checks. Verifierad data, men presenteras med sin faktiska aktualitet.
Stale	Data vars aktualitet inte längre kan styrkas. Stoppar operational motion; visas med badge och senast bekräftad tid.
Unknown	Tillstånd när tillräcklig information saknas. Neutral/violettgrå, ingen motion; mappas

Term	Betydelse
	aldrig till idle eller completed.
Drilldown	Navigering in i en nods interna struktur med bibehållet ankare, breadcrumb och back.
Isolate	Uttrycklig handling som döljer irrelevant information (projekt, community eller path) med tydlig exit.
Semantic zoom	Zoom som förändrar informationsnivån, inte bara skalan — fem nivåer från portfolio overview till execution detail.
Reduced motion	Läge som tar bort fysik, partiklar och pulser men behåller all statusinformation statiskt.
Graphify-artifact	Den publicerade artefakt som System Map bygger på; versionerad, sanitiserad, reproducerbar och med definierad fallback.
Activity rail	Textbaserad eventlista vid grafen; varje rad bygger på ett verifierat event och respekterar projektscope.
Inspector	Detaljpanel för vald nod: status, relationer, tider och behöriga actions.
Listvy	Fullvärdig parallell list-/tabellrepresentation som delar selection, filter, search, scope och status med grafen.
Pause motion	Kontroll som stoppar dekorativ rörelse och simulation utan att stoppa datauppdateringar.
Freeze live view	Kontroll som fryser den synliga runtime-vyn medan nya events buffras säkert.
Design token	Semantiskt namngiven visuell parameter (färg, storlek, glow m.m.) som styr utseendet utan att ändra graph contract.
Graph contract	Det sanitiserade datakontraktet mellan backend och graf: nodtyper, edgetyper, statusfält, scope och metadata.
Source of truth	Den auktoritativa datakällan per entity type, dokumenterad i ett source of truth-register.
RLS	Row Level Security — databasens radnivåskydd som backend-enforcerar projektsolering.
Codex-review	Obligatorisk extern granskning av den betydande diffen före merge.
Event envelope	Det gemensamma eventkontraktet med ID, typ, tidsstämplar, scope, correlation/causation och schema version.
Correlation-ID	Binder ihop events som tillhör samma övergripande execution.
Causation-ID	Anger vilket event eller beslut som orsakade nästa event.
Quality levels	Kvalitetsnivåerna High, Medium, Low och Static (med Auto som standard) som degraderar presentation före information.

Beroenden mellan kapitlen

Boken är skriven i beroendeordning: varje kapitel bygger på de föregående och förser de efterföljande med definitioner.

Kapitel	Bygger på	Levererar till efterföljande kapitel
1 Produktvision	—	tre huvudlägen, hierarki, projektmodell, tio designprinciper
2 Motion design	1	motion-nivåer, globala tillstånd, Atlas rörelsespråk, motion states
3 Visuellt språk	1, 2	former, storlekar, statuslager, edge-semantik, färgtokens
4 Zoom och interaktion	1–3	semantic zoom, labelmotor, selection, drilldown, isolate, filter
5 Realtime och sanning	1–4	datakategorier, event envelope, livscyklar, freshness, isolering
6 Tillgänglighet	2–5	listvy, tangentbord, screen reader, reduced motion, mobilstrategi
7 Performance budget	2–6	frame-/label-/motion-/event-/minnesbudgetar, kvalitetsnivåer
8 Implementationsfaser	1–7	sex faser, exit gates, stoppregler, artifact- och Replay-avgränsning
9 Testplan	1–8	testnivåer, datasets, fasvisa tester, severity, release gate
10 Git och leverans	8, 9	branch, commits, migrations, PR, Codex-review, merge gate, rollback

Centrala tvärgående beroenden:

- Operational motion (kapitel 2) förutsätter verifierade runtime-events (kapitel 5) — därför byggs Fas 3 efter Fas 1–2 (kapitel 8).
- Labelmotorn (kapitel 4) förutsätter label anchors och prioritetsskyltar (kapitel 3) och begränsas av labelbudgeten (kapitel 7).
- Listvyn (kapitel 6) delar selection- och statusmodell med grafen (kapitel 4) och är samtidigt fallback i prestandadegraderingen (kapitel 7).
- Projektisoleringen definieras produktmässigt i kapitel 1, tekniskt i kapitel 5, testas blockerande i kapitel 9 och skyddas i leveransen i kapitel 10.
- Execution Replay förutsätter den eventhistorik som realtime-strategin producerar (kapitel 5) och är uttryckligen utanför scope (kapitel 8).
- Graphify-artifacten analyseras i Fas 0 (kapitel 8), testas i kapitel 9 och får sitt leveransflöde i kapitel 10.

Implementeringssekvens

Den godkända sekvensen i sammandrag — detaljerna finns i kapitel 8:

1. **Fas 0 — Audit & contracts.** Verifiera repository, renderingsstack, graph contract, runtime-källor, säkerhet och Graphify-artifact. Ingen större produktionskod före granskad audit.
2. **Fas 1 — Visual foundation.** Design tokens, nod- och edge-renderers, Atlas identitet, projektterritorier och kontrollerad stabilisering — utan full realtime.
3. **Fas 2 — Zoom, labels & interaction.** Semantic zoom, labelmotor med collision avoidance, selection, drilldown, focus/isolate, kamera och URL-state.
4. **Fas 3 — Live Operations truth layer.** Snapshot, event normalization, hybrid realtime, verkliga operational states, connection states, stale/unknown, freeze och activity rail.
5. **Fas 4 — Accessibility, responsive & list view.** Synkroniserad listvy, tangentbord, screen reader, reduced motion, high contrast och mobil.
6. **Fas 5 — Scale & quality adaptation.** Aggregation, kvalitetsnivåer, automatisk degradering, render scheduler, eventuell Web Worker och extremdata tester.
7. **Fas 6 — Hardening & release.** Regression, säkerhet, prestanda, accessibility, visuell QA, Codex-review och PR enligt kapitel 10.

Varje fas verifieras enligt kapitel 9 innan nästa startar. Stoppreglerna i kapitel 8 och 10 gäller genomgående.

Slutlig acceptans och styrning

Arkitektoniska invarianter

Följande invarianter gäller över hela arkitekturen och får inte försvagas av någon implementation, något tema eller någon optimering:

Operational truth before spectacle. Ingen animation utan verifierat runtime-stöd; osäkerhet visas som stale, unknown, degraded eller reconnecting.

Projektilisering är backend-enforced. Klienten tar aldrig emot andra projekts privata data; RLS och subscription-filtrering sker på serversidan.

Failure och incident är skilda. Failure är ett immutabelt faktum; incidentens resolution skriver aldrig om failure-historiken. Retries skapar nya attempts.

Approvals kräver explicit, serverbekräftad action. Aldrig genom nodklick eller gest; egen livscykel; historiken bevaras efter beslut.

Tillgänglighet är ett parallellt gränssnitt. Listvy, tangentbord, screen reader och reduced motion bär samma information som canvasen.

Degradera presentation före information. Kritisk statusinformation överlever varje kvalitets- och prestandaoptimering.

Stabil mental karta. Hover, selection, inspector och små events räknar aldrig om layouten; visuell semantik betyder samma sak i alla vyer.

History remains history. Historiska events ser aldrig live ut; completed-animationer spelas endast inom aktualitetsfönstret.

Implementationsberoenden

Implementationen förutsätter, i ordning: verifierade kontrakt och source of truth-register (Fas 0); tokenbaserad visuell grund (Fas 1); label- och interaktionsmotor (Fas 2); verifierat truth layer (Fas 3); parallella gränssnitt (Fas 4); skalning (Fas 5); samt hardening med Codex-review (Fas 6). Migrationer skapas endast vid verifierat behov, och den befintliga MVP:n är plattformen — inte en omskrivning.

Uttryckliga icke-mål

Arbetet ska inte: bygga ett generellt visualiseringsverktyg; visa alla historiska runs samtidigt; göra alla noder konstant rörliga; skapa falska demosignaler; bygga full Execution Replay; göra mobilen identisk med desktop; ersätta vanliga dashboards och loggar; låta grafen ändra runtime-status utan korrekt backend-action; bygga permanent manuell layoutredigering; ändra projektiliseringens säkerhetsmodell; rendera 10 000 noder samtidigt; ge varje event en egen animation; behålla alla preliminära gränsvärden efter verklig profilering.

Release gates

En release är tillåten endast när samtliga gates i kapitel 9 och 10 är uppfyllda, sammanfattat: inga öppna blocker- eller ohanterade critical-fel; build och obligatoriska tester passerar; projektisolering och RLS backend-verifierade; ingen falsk runtime-status; approvals serverbekräftade; ingen okontrollerad minnesökning; primär desktop (MacBook Air M4) och primär mobil (Pixel 10 Pro) verifierade; accessibility-huvudflöden fungerar; Codex-review hanterad; testbevis sparade; PR med risker och rollbackplan; grundarens godkännande före merge.

Uppskjutna förmågor

Följande är medvetet uppskjutna och ingår inte i nuvarande scope: full Execution Replay; permanent manuell nod-pinning; presentation mode; manuell kvalitetskontroll för slutanvändare (Auto är standard tills vidare); cachead snapshot-visning vid initial load; projekt-health-betyg utan definierat backend-kontrakt; Web Worker utan mätbart behov; ytterligare runtime-statusar (retrying, paused, blocked, scheduled, skipped, degraded) tills de verifierats i verkliga kontrakt.

Förutsättningar för Execution Replay

Replay aktiveras som separat epic först när Live Operations är stabilt och följande finns: persisted events med ordering; correlation och causation; attempts-historik; approval history; incident history; output links; execution detail view. Replay använder historiskt verifierade events — aldrig klientens animationslogg — och all playback märks tydligt som historisk.

Kanoniseringsprocess

Denna bok har status **CANONICAL — FOUNDER APPROVED**. Grundaren godkände den slutliga PDF:en den 11 juli 2026. Kanoniseringen genomfördes enligt följande steg:

1. Grundaren granskar den slutliga PDF:en i sin helhet.
2. Eventuella redaktionella korrigeringar förs in; arkitektoniska ändringar kräver att berört källdokument först revideras och godkänns.
3. Grundaren godkänner boken uttryckligen.
4. Statusen uppdateras till CANONICAL med versionsnummer och datum; boken blir därefter den samlade referensen för Intelligence Graph-arkitekturen.
5. Senare avvikelser dokumenteras öppet; de godkända styrdokumentet och boken skrivs aldrig om i efterhand för att dölja avvikelser.

Källmanifest

Boken bygger uteslutande på följande tio godkända källdokument i `architecture/intelligence-graph/`. Inga arkitektoniska beslut har lagts till, tagits bort eller förändrats.

Nr	Källdokument	Version	Kapitel	Godkänt	Redaktionella korrigeringar
1	Product Vision v1.0.docx	1.0	1	Ja	Ja: rubriknormalisering, borttagen utgången statusrubrik
2	MOTION DESIGN SPECIFICATION v1.0.docx	1.0	2	Ja	Ja: rubriknormalisering, terminologi (activity rail)
3	Node & Edge Visual Language v1.0.docx	1.0	3	Ja	Ja: rubriknormalisering, tabellkonsolidering
4	Zoom, Label & Interaction Strategy v1.0.docx	1.0	4	Ja	Ja: borttagna formulärartefakter, terminologi (activity rail)
5	REALTIME & OPERATIONAL TRUTH STRATEGY v1.0.docx	1.0	5	Ja	Ja: rubriknormalisering
6	ACCESSIBILITY & RESPONSIVE STRATEGY v1.0.docx	1.0	6	Ja	Ja: rubriknormalisering, konsoliderade tangentbordsavsnitt
7	PERFORMANCE BUDGET v1.0.docx	1.0	7	Ja	Ja: rubriknormalisering, tabellkonsolidering
8	Implementation Phases v1.0.docx	1.0	8	Ja	Ja: borttagna formulärartefakter, rubriknormalisering
9	TEST PLAN v1.0.docx	1.0	9	Ja	Ja: rubriknormalisering
10	GIT & DELIVERY PLAN v1.0.docx	1.0	10	Ja	Ja: rubriknormalisering

Upptäckta källdefekter: två dokument (nr 4 och 8) avslutades med kvarlämnade formulärmarkörer ("Formulärets överkant"/"Formulärets nederkant") — redaktionella artefakter utan arkitektonisk innebörd, borttagna i boken. Samtliga tio dokument bar utgångna "Draft for Review"-rubriker som behandlats som utdaterade dokumenthuvuden i enlighet med källstatusen. Inga trunkerade avslut, saknade rubriker eller ofullständiga meningar hittades. Implementation

Phases v1.0 bekräftades innehålla det kompletta korrigerade avslutet med Graphify-artifactens leverans, fasgränser och stoppregler samt övergripande acceptanskriterier.

Källdokumenten har inte modifierats. Denna bok ersätter inte källdokumenten förrän den är kanoniserad enligt processen ovan.